DESARROLLO MÓVIL · INGENIERÍA EN SOFTWARE · GUÍA 1 DE 2: CONSTRUYE ESTA PRIMERO

Construye la API y el sitio web de la Biblioteca

Tutorial paso a paso con Next.js 16, Firebase y TypeScript. Unidad 03: semanas 9 a 12

Asignatura	Desarrollo Móvil (PREISESP17015)
Unidad	03 · Integración con API, seguridad y servicios del dispositivo
Qué construirás	Una API REST con Next.js que usa Firebase (autenticación y datos) y un sitio web con las mismas funciones que la app móvil
Orden	Esta guía primero (API y web); después la Guía 2 (app móvil con SQLite)
Tecnologías	Next.js 16 (App Router, Route Handlers), React 19, TypeScript, Firebase Admin SDK, Zod, Tailwind CSS, Vitest
Resultado	Proyecto `library_api` completo, con pruebas, que consume la app móvil

Contenido

1. Antes de empezar 4

1.1 Qué vas a construir 4

1.2 Cómo usar esta guía 6

1.3 Requisitos 6

1.4 Ruta de trabajo (API primero, después la app) 7

2. Paso 0 · Preparar el proyecto 8

2.1 Crear el proyecto de Next.js 8

2.2 Activar cacheComponents 8

2.3 Instalar dependencias 8

2.4 Scripts del proyecto 9

2.5 Firebase: lo nuevo para el servidor 9

3. Semana 9 · API REST con Route Handlers 11

3.1 Conceptos clave 11

3.2 Paso a paso 11

3.3 Cómo probar con datos reales 22

3.4 Evidencia y criterios 22

4. Semana 10 · Autenticación y autorización con Firebase 23

4.1 Conceptos clave 23

4.2 Paso a paso 23

4.3 Matriz de autorización 29

4.4 Cómo probar 29

4.5 Amenazas y controles 30

4.6 Evidencia y criterios 30

5. Semana 11 · Préstamos, sincronización y conflictos en el servidor 31

5.1 Paso a paso 31

5.2 Cómo probar 36

5.3 Evidencia y criterios 37

6. Semana 12 · Portadas: el servicio del dispositivo, visto desde el servidor 38

6.1 Paso a paso 38

6.2 Cómo probar 40

6.3 Evidencia y criterios 40

7. El sitio web 41

7.1 Sesión del sitio web 41

7.2 Componentes de interfaz 43

7.3 Estilos, layout y redirección 48

7.4 Las páginas 49

7.5 Cómo probar el sitio 53

8. Pruebas finales y restablecer datos 54

8.1 Restablecer los préstamos y las portadas 54

8.2 Prueba de humo contra Firebase real 54

8.3 Verificación completa 61

9. Seguridad y despliegue 62

9.1 Lista de verificación antes de publicar 62

9.2 Desplegar en Vercel (resumen) 62

10. Problemas frecuentes 63

11. Bitácora de uso de IA generativa 64

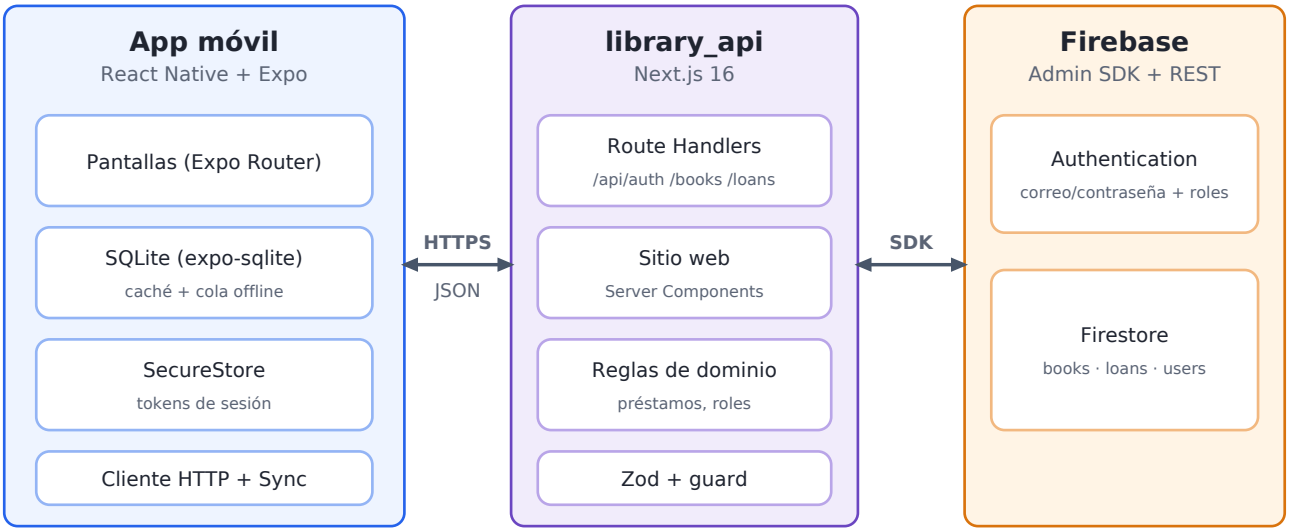
12. Referencias 65

1. Antes de empezar

1.1 Qué vas a construir

Un solo proyecto de Next.js que cumple dos papeles:

1. Una **API REST** (`/api/...`) que consumirá la app móvil: autenticación, catálogo de libros, préstamos y portadas.
2. Un **sitio web** con las mismas funciones que la app: iniciar sesión, listar libros, ver el detalle y solicitar préstamos (estudiante); gestionar préstamos y cambiar portadas (bibliotecario).



La app nunca habla con Firebase: todo pasa por la API.

Tres ideas explican todo el diseño:

- **La app móvil nunca habla con Firebase.** Solo conoce esta API. Las credenciales de administrador viven únicamente en el servidor.
- **Las reglas de negocio están en un solo lugar** (por ejemplo, "no se presta un libro sin ejemplares"), así que la web y la app se comportan igual.
- **El servidor es la fuente de verdad.** Cuando el teléfono y el servidor no coinciden, gana el servidor.

Roles y funcionalidades

Funcionalidad	Estudiante	Bibliotecario	Dónde
Crear cuenta (siempre como estudiante)	✓		App y web
Iniciar sesión	✓	✓	App y web
Listar libros, buscar y filtrar	✓	✓	App y web
Ver detalle del libro	✓	✓	App y web
Solicitar préstamo	✓		App y web
Ver mis préstamos	✓		App y web
Ver **todos** los préstamos		✓	App y web
Prestar, rechazar, registrar devolución		✓	App y web
Cambiar la portada de un libro		✓	App (cámara) y web (archivo)

No hay límite de usuarios en ningún rol. El catálogo se carga con **10 libros**.

Endpoints que vas a crear

Método y ruta	Quién	Qué hace	Semana
`GET /api/health`	Público	Comprueba que el servidor responde	9
`GET /api/books?q=&category=&updatedSince=`	Autenticado	Lista de libros; `updatedSince` devuelve solo los modificados	9 y 10
`GET /api/books/:id`	Autenticado	Detalle de un libro	9 y 10
`POST /api/auth/register`	Público	Crea un estudiante y devuelve tokens	10
`POST /api/auth/login`	Público	Inicia sesión	10
`POST /api/auth/refresh`	Público	Cambia un `refreshToken` por un `idToken` nuevo	10
`GET /api/auth/me`	Autenticado	Usuario actual y su rol	10
`GET /api/loans?status=`	Autenticado	Estudiante: los suyos. Bibliotecario: todos	11
`POST /api/loans`	Estudiante	Solicita un préstamo (idempotente)	11
`GET /api/loans/:id`	Autenticado	Un préstamo	11
`PATCH /api/loans/:id`	Bibliotecario	`approve`, `reject` o `return`	11
`GET /api/books/:id/cover`	Público	Imagen de portada	12
`PUT /api/books/:id/cover`	Bibliotecario	Sube la portada	12

Todos los errores tienen la misma forma, para que la app reaccione al `code` y no al texto del mensaje:

```
{ "error": { "code": "NO_COPIES", "message": "No hay ejemplares disponibles" } }
```

Datos en Firestore

Colección	Documento	Campos principales
`books`	`cien-anos-de-soledad` (slug)	`title`, `author`, `category`, `year`, `description`, `totalCopies`, `availableCopies`, `coverVersion`, `updatedAt`
`bookCovers`	mismo id del libro	`contentType`, `data` (base64), `updatedAt`
`loans`	`{uid}_{clientRequestId}`	`bookId`, `bookTitle`, `userId`, `userEmail`, `userName`, `status`, `clientRequestId`, `requestedAt`, `resolvedAt`, `dueDate`, `returnedAt`, `updatedAt`
`users`	`{uid}`	`email`, `displayName`, `role`, `createdAt`

Estados de un préstamo: `requested` → `approved` → `returned`, o `requested` → `rejected`.

1.2 Cómo usar esta guía

Esta guía es un **tutorial de construcción**: al terminarla tienes el proyecto completo y funcionando. No hay archivos "que se dan por hechos". Cada paso tiene una etiqueta de color:

Etiqueta	Qué debes hacer
CREAR	Crea el archivo en esa ruta (relativa a la raíz del proyecto) con **exactamente** ese contenido. Si ya existe, reemplázalo completo.
AGREGAR A	Agrega ese fragmento JSON a un archivo que ya existe (sin borrar lo demás).
AGREGAR AL FINAL DE	Pega esas líneas al final del archivo.
ELIMINAR	Borra ese archivo.
TERMINAL	Ejecuta esos comandos dentro de la carpeta indicada.

Después de cada semana hay un **punto de control**: ejecuta `pnpm typecheck` y `pnpm test`. Si pasan, sigues; si no, revisa el último archivo que creaste (un error de TypeScript suele indicar un archivo que falta o una letra mal copiada).

Idea. Trabaja con Git: haz un commit al terminar cada semana (`git add -A && git commit -m "semana 9"`). Si algo se rompe, `git diff` te muestra qué cambió.

Nota. Los comentarios dentro del código están en español y explican el porqué de cada decisión. Léelos: son parte de la guía.

1.3 Requisitos

- **Node.js 24 LTS** (mínimo 22.5) y **pnpm** (`npm install -g pnpm`). Git y Visual Studio Code.
- Un **proyecto de Firebase** con Authentication (correo y contraseña) y Firestore habilitados. Ya sabes crearlos; en la sección 2.5 solo configuramos lo nuevo para el servidor.
- Una terminal tipo **bash** (macOS, Linux, WSL o Git Bash en Windows). Los ejemplos usan `curl`; en PowerShell usa `curl.exe` o el equivalente `Invoke-RestMethod`.

1.4 Ruta de trabajo (API primero, después la app)

Construye **toda esta guía** (semanas 9 a 12 y el sitio web) y después la **Guía 2** (app móvil). Si en clase avanzan semana a semana, haz primero la sección de la API de esa semana y luego la de la app:

Semana	Guía 1 (API y web)	Guía 2 (app móvil)	Lo que ya funciona al terminar
9	Capítulo 3: REST, validación, catálogo	Capítulo 3: cliente HTTP y estados de pantalla	La app lista libros desde la API
10	Capítulo 4: Firebase Auth, roles	Capítulo 4: sesión segura y rutas protegidas	Registro, login y roles
11	Capítulo 5: préstamos, idempotencia, conflictos	Capítulo 5: SQLite offline y sincronización	Solicitudes sin conexión
12	Capítulo 6: portadas	Capítulo 6: cámara y galería	**Hito 3**
—	Capítulo 7: sitio web · Capítulo 8: pruebas finales	Capítulo 7: pruebas de integración	Entrega

2. Paso 0 · Preparar el proyecto

2.1 Crear el proyecto de Next.js

```
pnpm create next-app@latest library_api
```

Acepta las opciones recomendadas: **TypeScript**, **ESLint**, **Tailwind CSS**, **App Router**, **Turbopack**, **sin** carpeta `src/` y alias de importación `@/*`. Entra a la carpeta (`cd library_api`) y comprueba que la versión de `next` en `package.json` sea la **16**.

> **Ojo.** Esta guía se escribió y se comprobó con Next.js 16.4. Next cambia rápido: si ves una versión mayor, revisa la documentación incluida en `node_modules/next/dist/docs/` antes de copiar código.

2.2 Activar `cacheComponents`

Abre `next.config.ts` y asegúrate de que `nextConfig` tenga la línea `cacheComponents: true` (conserva cualquier otra opción que ya traiga tu plantilla):

```ts
const nextConfig: NextConfig = {
  cacheComponents: true,
  // ...el resto de tu configuración
};
```

Con `cacheComponents` activado, todo lo que depende de la petición (cookies, parámetros de búsqueda) debe leerse **dentro de** `<Suspense>`; lo verás en el capítulo del sitio web. Otro cambio de Next 16: el middleware ahora se llama `proxy.ts`.

2.3 Instalar dependencias

Antes de instalar, una advertencia: pnpm 11 exige **decidir** qué hacer con los scripts de instalación de ciertos paquetes (`firebase-admin` trae algunos). Si no lo decides, `pnpm add` termina con el error `ERR_PNPM_IGNORED_BUILDS`. Los de este proyecto son opcionales, así que los dejamos rechazados explícitamente en `pnpm-workspace.yaml` (reemplaza el contenido del archivo):

▸ **CREAR · library_api/pnpm-workspace.yaml**

```yaml
allowBuilds:
  "@firebase/util": false
  esbuild: false
  protobufjs: false
  sharp: false
  unrs-resolver: false
```

Ahora sí, instala:

▸ **TERMINAL · library_api**

```
pnpm add firebase-admin@14 zod@4 server-only
pnpm add -D vitest@5 tsx@4
```

- **firebase-admin**: el SDK de administrador de Firebase (solo corre en el servidor).
- **zod**: validación de entradas.
- **server-only**: hace que el build falle si por error importas código del servidor desde un componente del navegador.
- **vitest**: pruebas. **tsx**: ejecuta scripts de TypeScript (para cargar datos).

2.4 Scripts del proyecto

Agrega estos scripts a la sección `"scripts"` de `package.json` (conserva los que ya existen: `dev`, `build`, `start`, `lint`):

▸ **AGREGAR A · library_api/package.json**

```
{
  "scripts": {
    "dev:lan": "next dev -H 0.0.0.0",
    "typecheck": "next typegen && tsc --noEmit",
    "test": "vitest run",
    "seed": "tsx --env-file=.env scripts/seed.mts",
    "set-role": "tsx --env-file=.env scripts/set-role.mts",
    "create-user": "tsx --env-file=.env scripts/create-user.mts",
    "smoke": "tsx --env-file=.env scripts/smoke.mts",
    "reset-demo": "tsx --env-file=.env scripts/reset-demo.mts"
  }
}
```

- `dev:lan` hace que el servidor acepte conexiones de tu red local (necesario para usarlo desde el teléfono).
- `typecheck` primero genera los tipos de rutas de Next (`RouteContext`, `PageProps`) y luego revisa TypeScript.
- Los scripts `.mts` los escribiremos más adelante; el sufijo `.mts` los marca como módulos ES, necesarios para usar `await` al nivel superior.

2.5 Firebase: lo nuevo para el servidor

Ya sabes crear el proyecto, habilitar Authentication → Correo/contraseña y crear Firestore. Verifica que ambos estén activos. Lo nuevo es que **el servidor** necesita credenciales de administrador:

1. **Cuenta de servicio:** Consola de Firebase → Configuración del proyecto → Cuentas de servicio → **Generar nueva clave privada**. Se descarga un `.json` del que necesitas `project_id`, `client_email` y `private_key`.
2. **Clave de API web:** Configuración del proyecto → General → Tus apps → app web (créala con `</>` si no tienes una) → `apiKey`. La usaremos para iniciar sesión con la API REST de Firebase Authentication.
3. Crea el archivo `.env` (ya lo ignora git) con tus valores:

```
FIREBASE_PROJECT_ID=tu-proyecto
FIREBASE_WEB_API_KEY=AIza...
FIREBASE_CLIENT_EMAIL=firebase-adminsdk-xxxxx@tu-proyecto.iam.gserviceaccount.com
# Entre comillas dobles y con los saltos de línea escritos como \n
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"
```

4. Crea también la plantilla `.env.example` (esta **sí** se sube a git; solo tiene valores de ejemplo):

▸ **CREAR · library_api/.env.example**

```
# Copia este archivo a .env (o .env.local) y completa los valores.
# Firebase Console → Configuración del proyecto → Cuentas de servicio → "Generar nueva clave privada"
FIREBASE_PROJECT_ID=tu-proyecto
FIREBASE_CLIENT_EMAIL=firebase-adminsdk-xxxxx@tu-proyecto.iam.gserviceaccount.com
# Entre comillas dobles y con los saltos de línea escritos como \n
FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"
# Firebase Console → Configuración del proyecto → General → "Clave de API web"
FIREBASE_WEB_API_KEY=AIza...
```

5. Evita que las credenciales lleguen a git, pegando esto al final de `.gitignore`:

▸ **AGREGAR AL FINAL DE · library_api/.gitignore**

```
!.env.example

# Firebase service account keys (credenciales de administrador: NUNCA subir a git)
*-firebase-adminsdk-*.json
serviceAccount*.json
```

Ojo. El `.json` de la cuenta de servicio es una **credencial de administrador total** de tu proyecto. Cuando copies sus tres valores al `.env`, **mueve o borra el archivo** de la carpeta del proyecto. Si alguna vez se sube a un repositorio público, bórralo desde la consola de Firebase y genera otro.

6. Como solo el servidor accede a Firestore (la cuenta de servicio se salta las reglas), las reglas deben **negar todo** acceso directo desde clientes. En Firestore → Reglas:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

Nota. Punto de control 0. `pnpm typecheck` debe terminar sin errores. Aún no hay código nuestro, pero confirma que el proyecto y TypeScript funcionan.

3. Semana 9 · API REST con Route Handlers

Del microcurrículo: Consumo de API REST: HTTP, JSON, asincronía, configuración por ambientes y estados de carga, vacío, error y reintento. Esta es la contraparte del servidor: el lado que **responde** lo que la app consume.

3.1 Conceptos clave

- **Route Handler.** Un archivo `route.ts` dentro de `app/` que exporta funciones con el nombre del método HTTP (`GET`, `POST`, `PATCH`...). La ruta del archivo es la URL: `app/api/books/[id]/route.ts` atiende `/api/books/clean-code`.
- **Código de estado.** `200` éxito · `201` creado · `400` datos inválidos · `401` no autenticado · `403` sin permiso · `404` no existe · `409` conflicto con el estado actual · `500` error del servidor.
- **Nunca confíes en la entrada.** Todo lo que llega (cuerpo, parámetros, cabeceras) se valida antes de usarse.
- **Errores con forma estable.** La app decide qué hacer según `code` (`NO_COPIES`), no según el texto del mensaje.

3.2 Paso a paso

Paso 9.1 · Los tipos del dominio

Todo el proyecto comparte estos tipos. Piensa en ellos como el contrato entre la API, el sitio web y la app.

▸ **CREAR · library_api/lib/types.ts**

```ts
export type Role = "student" | "librarian";

export interface AuthUser {
  uid: string;
  email: string;
  name: string;
  role: Role;
}

export interface Book {
  id: string;
  title: string;
  author: string;
  category: string;
  year: number;
  description: string;
  totalCopies: number;
  availableCopies: number;
  /** Ruta relativa a la API (ej. /api/books/abc/cover?v=2) o null si no hay portada. */
  coverUrl: string | null;
  /** ISO 8601. Se usa como cursor para la sincronización incremental (?updatedSince=). */
  updatedAt: string;
}

export type LoanStatus = "requested" | "approved" | "rejected" | "returned";

export interface Loan {
  id: string;
  bookId: string;
  bookTitle: string;
  userId: string;
  userEmail: string;
  userName: string;
```

```
  status: LoanStatus;
  /** Llave de idempotencia generada por el cliente (semana 11: cola offline). */
  clientRequestId: string;
  requestedAt: string;
  resolvedAt: string | null;
  dueDate: string | null;
  returnedAt: string | null;
  updatedAt: string;
}

export type LoanAction = "approve" | "reject" | "return";

export interface TokenSet {
  idToken: string;
  refreshToken: string;
  /** Segundos de vigencia del idToken. */
  expiresIn: number;
}
```

Paso 9.2 · Errores con código estable

`ApiError` es el error "de negocio": lleva el estado HTTP y un `code` que la app puede interpretar. Lo lanzaremos desde cualquier capa y un único envoltorio (paso 9.5) lo convertirá en respuesta.

▸ **CREAR · library_api/lib/errors.ts**

```
export type ErrorCode =
  | "VALIDATION_ERROR"
  | "UNAUTHENTICATED"
  | "INVALID_CREDENTIALS"
  | "FORBIDDEN"
  | "NOT_FOUND"
  | "EMAIL_IN_USE"
  | "ALREADY_ACTIVE"
  | "NO_COPIES"
  | "INVALID_STATE"
  | "PAYLOAD_TOO_LARGE"
  | "RATE_LIMITED"
  | "CONFIG_ERROR"
  | "INTERNAL_ERROR";

/** Error de negocio con status HTTP y código estable que la app móvil puede interpretar. */
export class ApiError extends Error {
  constructor(
    readonly status: number,
    readonly code: ErrorCode,
    message: string,
    readonly details?: unknown,
  ) {
    super(message);
    this.name = "ApiError";
  }
}
```

Paso 9.3 · Validar entradas con Zod

Cada endpoint declara el esquema de lo que acepta. Si no se cumple, se responde `400` con un mensaje en español. Este archivo reúne los esquemas de todo el proyecto; los de préstamos y portadas los usarás en las semanas 11 y 12.

▸ **CREAR · library_api/lib/schemas.ts**

```
import { z } from "zod";

// Mensajes de validación por defecto en español (ej. "Invalid input" → "Entrada inválida").
z.config(z.locales.es());

/** Primero se normaliza (los teclados móviles añaden espacios al autocompletar) y luego se valida. */
const emailField = z.string().trim().toLowerCase().pipe(z.email("Correo inválido"));

export const loginSchema = z.object({
  email: emailField,
  password: z.string().min(1, "La contraseña es obligatoria"),
});
```

```
export const registerSchema = z.object({
  email: emailField,
  password: z.string().min(6, "La contraseña debe tener al menos 6 caracteres"),
  displayName: z.string().trim().min(2, "El nombre debe tener al menos 2 caracteres").max(80),
});

export const refreshSchema = z.object({
  refreshToken: z.string().min(1),
});

export const createLoanSchema = z.object({
  bookId: z.string().min(1).max(120),
  clientRequestId: z
    .string()
    .min(8)
    .max(100)
    .regex(/^[A-Za-z0-9_-]+$/, "Formato de clientRequestId inválido"),
});

export const loanActionSchema = z.object({
  action: z.enum(["approve", "reject", "return"]),
});

/** Límite de la portada en base64: Firestore admite 1 MiB por documento. */
export const MAX_COVER_BASE64_LENGTH = 700_000;

export const coverSchema = z.object({
  contentType: z.enum(["image/jpeg", "image/png", "image/webp"]),
  data: z
    .string()
    .min(100)
    .max(MAX_COVER_BASE64_LENGTH, "La imagen es demasiado grande")
    .regex(/^[A-Za-z0-9+/]+={0,2}$/, "Base64 inválido"),
});

export const bookQuerySchema = z.object({
  q: z.string().trim().max(100).optional(),
  category: z.string().trim().max(60).optional(),
  updatedSince: z.iso.datetime().optional(),
});

export const loanQuerySchema = z.object({
  status: z.enum(["requested", "approved", "rejected", "returned"]).optional(),
});
```

Detalles que evitan errores reales:

- `z.config(z.locales.es())` traduce los mensajes por defecto de Zod.
- El correo se **normaliza antes de validarse** (recortar espacios y pasar a minúsculas). Los teclados móviles añaden un espacio al autocompletar; validar antes de recortar rechazaría correos válidos.
- `clientRequestId` solo admite letras, números, `-` y `_`: se usa para armar el id de un documento y no debe permitir caracteres como `/`.

Paso 9.4 · Variables de entorno y conexión con Firebase

Las variables se validan **la primera vez que se usan** (no al compilar), para que `pnpm build` no exija secretos. Si falta alguna, el error dice exactamente cuáles:

▸ **CREAR · library_api/lib/env.ts**

```
import { z } from "zod";
import { ApiError } from "./errors";

const envSchema = z.object({
  FIREBASE_PROJECT_ID: z.string().min(1),
  FIREBASE_CLIENT_EMAIL: z.string().min(1),
  FIREBASE_PRIVATE_KEY: z.string().min(1),
  /** "Web API Key" de la configuración del proyecto Firebase (para Auth REST). */
  FIREBASE_WEB_API_KEY: z.string().min(1),
```

```
});

export type Env = z.infer<typeof envSchema>;

let cached: Env | null = null;

/** Se valida de forma perezosa para que `next build` no exija secretos. */
export function getEnv(): Env {
  if (cached) return cached;
  const parsed = envSchema.safeParse(process.env);
  if (!parsed.success) {
    const missing = parsed.error.issues.map((i) => i.path.join("."));
    throw new ApiError(
      500,
      "CONFIG_ERROR",
      `Faltan variables de entorno: ${missing.join(", ")}. Revisa .env`,
    );
  }
  cached = {
    ...parsed.data,
    // En .env las saltos de línea de la llave privada llegan escapadas como \n
    FIREBASE_PRIVATE_KEY: parsed.data.FIREBASE_PRIVATE_KEY.replace(/\\n/g, "\n"),
  };
  return cached;
}
```

`firebase.ts` inicia el Admin SDK una sola vez y expone `adminAuth()` y `db()`. El `import "server-only"` impide usarlo desde el navegador:

▸ **CREAR · library_api/lib/firebase.ts**

```
import "server-only";
import { cert, getApp, getApps, initializeApp, type App } from "firebase-admin/app";
import { getAuth } from "firebase-admin/auth";
import { getFirestore } from "firebase-admin/firestore";
import { getEnv } from "./env";

function adminApp(): App {
  if (getApps().length > 0) return getApp();
  const env = getEnv();
  return initializeApp({
    credential: cert({
      projectId: env.FIREBASE_PROJECT_ID,
      clientEmail: env.FIREBASE_CLIENT_EMAIL,
      privateKey: env.FIREBASE_PRIVATE_KEY,
    }),
  });
}

export const adminAuth = () => getAuth(adminApp());
export const db = () => getFirestore(adminApp());
```

Paso 9.5 · Un envoltorio para unificar errores

Todo Route Handler se envuelve en `route()`: convierte cualquier excepción en la respuesta JSON uniforme y evita que se filtren detalles internos en los errores inesperados. También validamos aquí el cuerpo y los parámetros.

▸ **CREAR · library_api/lib/http.ts**

```
import { unstable_rethrow } from "next/navigation";
import { NextResponse } from "next/server";
import { ZodError, type ZodType } from "zod";
import { ApiError } from "./errors";

/** Convierte cualquier excepción en la respuesta JSON uniforme { error: { code, message } }. */
export function errorResponse(err: unknown): NextResponse {
  if (err instanceof ApiError) {
    return NextResponse.json(
      { error: { code: err.code, message: err.message, details: err.details } },
      { status: err.status },
    );
  }
  if (err instanceof ZodError) {
```

```
    return NextResponse.json(
      {
        error: {
          code: "VALIDATION_ERROR",
          message: err.issues[0]?.message ?? "Datos inválidos",
          details: err.issues.map((i) => ({ path: i.path.join("."), message: i.message })),
        },
      },
      { status: 400 },
    );
  }
  console.error("[api] error no controlado:", err);
  return NextResponse.json(
    { error: { code: "INTERNAL_ERROR", message: "Error interno del servidor" } },
    { status: 500 },
  );
}

type Handler<C> = (request: Request, ctx: C) => Promise<Response>;

/** Envuelve un Route Handler para unificar el manejo de errores. */
export function route<C = unknown>(handler: Handler<C>): Handler<C> {
  return async (request, ctx) => {
    try {
      return await handler(request, ctx);
    } catch (err) {
      // Next usa excepciones internas (p. ej. al leer request.headers durante el prerender):
      // no son errores de la aplicación y deben llegar intactas al framework.
      unstable_rethrow(err);
      return errorResponse(err);
    }
  };
}

/** Lee y valida el cuerpo JSON de la petición. */
export async function parseBody<T>(request: Request, schema: ZodType<T>): Promise<T> {
  let raw: unknown;
  try {
    raw = await request.json();
  } catch {
    throw new ApiError(400, "VALIDATION_ERROR", "El cuerpo debe ser JSON válido");
  }
  return schema.parse(raw);
}

/** Valida los query params de la URL. */
export function parseQuery<T>(request: Request, schema: ZodType<T>): T {
  const params = Object.fromEntries(new URL(request.url).searchParams);
  return schema.parse(params);
}
```

> **Ojo.** La línea `unstable_rethrow(err)` es imprescindible. Next.js usa **excepciones internas** como mecanismo de control (por ejemplo, al leer `request.headers` durante el prerender). Si un `catch` general las captura, la ruta se rompe en silencio. Este error se descubrió al compilar: el log mostraba `NEXT_PRERENDER_INTERRUPTED` tratado como un error 500.

Paso 9.6 · Acceso a los libros en Firestore

Esta capa es la única que conoce Firestore para los libros. Fíjate en tres decisiones:

- La búsqueda **ignora mayúsculas y tildes** (`colera` encuentra cólera).
- `updatedSince` filtra en Firestore por la fecha de modificación: es el cursor de la sincronización de la semana 11.
- `saveCover` y `getCover` los usarás en la semana 12; ya quedan aquí para tener el archivo completo.

▸ **CREAR · library_api/lib/data/books.ts**

```
import "server-only";
import { db } from "../firebase";
import { ApiError } from "../errors";
import type { Book } from "../types";

export const BOOKS = "books";
export const COVERS = "bookCovers";

type BookDoc = Omit<Book, "id" | "coverUrl"> & { coverVersion?: number };

export function toBook(id: string, data: BookDoc): Book {
  const { coverVersion = 0, ...rest } = data;
  return {
    id,
    ...rest,
    coverUrl: coverVersion > 0 ? `/api/books/${id}/cover?v=${coverVersion}` : null,
  };
}

export interface BookFilter {
  q?: string;
  category?: string;
  updatedSince?: string;
}

const normalize = (s: string) =>
  s
    .normalize("NFD")
    .replace(/\p{Diacritic}/gu, "")
    .toLowerCase();

export async function listBooks(filter: BookFilter = {}): Promise<Book[]> {
  let query: FirebaseFirestore.Query = db().collection(BOOKS);
  // Filtro incremental para la sincronización (semana 11): solo lo que cambió.
  if (filter.updatedSince) query = query.where("updatedAt", ">", filter.updatedSince);
  const snap = await query.get();

  let books = snap.docs.map((d) => toBook(d.id, d.data() as BookDoc));
  if (filter.category) books = books.filter((b) => b.category === filter.category);
  if (filter.q) {
    const q = normalize(filter.q);
    books = books.filter((b) => normalize(`${b.title} ${b.author}`).includes(q));
  }
  return books.sort((a, b) => a.title.localeCompare(b.title, "es"));
}

export async function getBook(id: string): Promise<Book> {
  const snap = await db().collection(BOOKS).doc(id).get();
  if (!snap.exists) throw new ApiError(404, "NOT_FOUND", "Libro no encontrado");
  return toBook(snap.id, snap.data() as BookDoc);
}

export async function listCategories(): Promise<string[]> {
  const books = await listBooks();
  return [...new Set(books.map((b) => b.category))].sort((a, b) => a.localeCompare(b, "es"));
}

export async function saveCover(bookId: string, contentType: string, data: string): Promise<Book> {
  const firestore = db();
  const bookRef = firestore.collection(BOOKS).doc(bookId);
  const coverRef = firestore.collection(COVERS).doc(bookId);

  await firestore.runTransaction(async (tx) => {
    const snap = await tx.get(bookRef);
    if (!snap.exists) throw new ApiError(404, "NOT_FOUND", "Libro no encontrado");
    const current = (snap.data() as BookDoc).coverVersion ?? 0;
    const now = new Date().toISOString();
    tx.set(coverRef, { contentType, data, updatedAt: now });
    // Subir updatedAt hace que el cambio llegue a los clientes en la próxima sincronización.
    tx.update(bookRef, { coverVersion: current + 1, updatedAt: now });
  });
  return getBook(bookId);
}

export async function getCover(bookId: string): Promise<{ contentType: string; bytes: Buffer }> {
  const snap = await db().collection(COVERS).doc(bookId).get();
```

```
  if (!snap.exists) throw new ApiError(404, "NOT_FOUND", "El libro no tiene portada");
  const { contentType, data } = snap.data() as { contentType: string; data: string };
  return { contentType, bytes: Buffer.from(data, "base64") };
}
```

Paso 9.7 · Las rutas del catálogo

Primero, una ruta de salud que no toca Firebase; sirve para comprobar que el servidor responde:

▸ **CREAR · library_api/app/api/health/route.ts**

```
/** Sirve para que la app compruebe que el servidor responde (sin tocar Firebase). */
export async function GET() {
  return Response.json({ status: "ok", service: "library_api" });
}
```

Ahora el catálogo. **En esta primera versión las rutas todavía no exigen sesión**; en la semana 10 las protegeremos y verás exactamente qué cambia.

▸ **CREAR · library_api/app/api/books/route.ts**

```
import { listBooks, listCategories } from "@/lib/data/books";
import { parseQuery, route } from "@/lib/http";
import { bookQuerySchema } from "@/lib/schemas";

/**
 * GET /api/books?q=&category=&updatedSince=
 * Versión de la semana 9: SIN autenticación (se agrega en la semana 10).
 * `serverTime` se toma ANTES de consultar para no perder cambios concurrentes.
 */
export const GET = route(async (request) => {
  const filter = parseQuery(request, bookQuerySchema);
  const serverTime = new Date().toISOString();
  const [books, categories] = await Promise.all([listBooks(filter), listCategories()]);
  return Response.json({ books, categories, serverTime });
});
```

▸ **CREAR · library_api/app/api/books/[id]/route.ts**

```
import { getBook } from "@/lib/data/books";
import { route } from "@/lib/http";

export const GET = route(async (_request, ctx: RouteContext<"/api/books/[id]">) => {
  const { id } = await ctx.params;
  return Response.json({ book: await getBook(id) });
});
```

Observa que la ruta solo valida, delega y responde. `RouteContext` es un tipo global que Next genera con `pnpm typecheck`. Y `serverTime` se toma **antes** de consultar: la app lo usará como cursor de la siguiente sincronización.

Paso 9.8 · Cargar los datos de ejemplo

Este script carga los **10 libros** en Firestore. Es seguro repetirlo: no pisa los ejemplares de libros existentes (con `--reset` los reescribe). **No crea usuarios**: así no queda ninguna cuenta con contraseña conocida escrita en el repositorio. Tus usuarios los creas tú en la semana 10 con `pnpm create-user`.

▸ **CREAR · library_api/scripts/seed.mts**

```
/**
 * Carga los 10 libros de ejemplo en Firestore.
 *   pnpm seed            → crea los que falten (no pisa ejemplares de libros existentes)
 *   pnpm seed -- --reset → vuelve a escribir los 10 libros desde cero
 *
 * NO crea usuarios: así no hay cuentas con contraseña conocida escrita en el repositorio.
 * Los usuarios los creas tú con `pnpm create-user` (ver la guía).
 */
import { cert, initializeApp } from "firebase-admin/app";
import { getFirestore } from "firebase-admin/firestore";
```

```
const required = ["FIREBASE_PROJECT_ID", "FIREBASE_CLIENT_EMAIL", "FIREBASE_PRIVATE_KEY"] as const;
for (const key of required) {
  if (!process.env[key]) {
    console.error(`Falta ${key}. Completa .env a partir de .env.example`);
    process.exit(1);
  }
}

initializeApp({
  credential: cert({
    projectId: process.env.FIREBASE_PROJECT_ID!,
    clientEmail: process.env.FIREBASE_CLIENT_EMAIL!,
    privateKey: process.env.FIREBASE_PRIVATE_KEY!.replace(/\\n/g, "\n"),
  }),
});

const db = getFirestore();
const reset = process.argv.includes("--reset");

interface SeedBook {
  id: string;
  title: string;
  author: string;
  category: string;
  year: number;
  description: string;
  totalCopies: number;
}

const books: SeedBook[] = [
  {
    id: "cien-anos-de-soledad",
    title: "Cien años de soledad",
    author: "Gabriel García Márquez",
    category: "Novela",
    year: 1967,
    description:
      "La historia de la familia Buendía a lo largo de siete generaciones en el pueblo ficticio de Macondo.",
    totalCopies: 3,
  },
  {
    id: "don-quijote",
    title: "Don Quijote de la Mancha",
    author: "Miguel de Cervantes",
    category: "Clásicos",
    year: 1605,
    description:
      "Un hidalgo enloquecido por los libros de caballerías sale a recorrer La Mancha junto a su escudero Sancho Panza.",
    totalCopies: 2,
  },
  {
    id: "amor-en-los-tiempos-del-colera",
    title: "El amor en los tiempos del cólera",
    author: "Gabriel García Márquez",
    category: "Novela",
    year: 1985,
    description: "Una historia de amor que espera más de cincuenta años en una ciudad del Caribe.",
    totalCopies: 2,
  },
  {
    id: "rayuela",
    title: "Rayuela",
    author: "Julio Cortázar",
    category: "Novela",
    year: 1963,
    description: "Novela que puede leerse en más de un orden, entre París y Buenos Aires.",
    totalCopies: 1,
  },
  {
    id: "pedro-paramo",
    title: "Pedro Páramo",
    author: "Juan Rulfo",
    category: "Novela",
    year: 1955,
    description:
      "Juan Preciado llega a Comala en busca de su padre y encuentra un pueblo poblado por voces de muertos.",
    totalCopies: 2,
```

```
  },
  {
    id: "la-ciudad-y-los-perros",
    title: "La ciudad y los perros",
    author: "Mario Vargas Llosa",
    category: "Novela",
    year: 1963,
    description: "La vida de un grupo de cadetes en un colegio militar de Lima.",
    totalCopies: 1,
  },
  {
    id: "clean-code",
    title: "Clean Code",
    author: "Robert C. Martin",
    category: "Tecnología",
    year: 2008,
    description:
      "Principios y prácticas para escribir código legible, mantenible y fácil de cambiar.",
    totalCopies: 3,
  },
  {
    id: "introduction-to-algorithms",
    title: "Introduction to Algorithms",
    author: "Cormen, Leiserson, Rivest y Stein",
    category: "Tecnología",
    year: 1990,
    description:
      "Referencia clásica de algoritmos y estructuras de datos con análisis de complejidad.",
    totalCopies: 2,
  },
  {
    id: "pragmatic-programmer",
    title: "The Pragmatic Programmer",
    author: "Andrew Hunt y David Thomas",
    category: "Tecnología",
    year: 1999,
    description:
      "Consejos prácticos para el oficio de desarrollar software a lo largo de una carrera.",
    totalCopies: 2,
  },
  {
    id: "dont-make-me-think",
    title: "Don't Make Me Think",
    author: "Steve Krug",
    category: "Diseño",
    year: 2000,
    description:
      "Una guía breve sobre usabilidad para que las interfaces se entiendan sin esfuerzo.",
    totalCopies: 2,
  },
];

async function seedBooks() {
  for (const { id, ...book } of books) {
    const ref = db.collection("books").doc(id);
    const exists = (await ref.get()).exists;
    if (exists && !reset) {
      console.log(`= libro existente: ${book.title}`);
      continue;
    }
    await ref.set({
      ...book,
      availableCopies: book.totalCopies,
      coverVersion: 0,
      updatedAt: new Date().toISOString(),
    });
    console.log(`+ libro: ${book.title}`);
  }
}

await seedBooks();
console.log("\nListo. Ahora crea tus usuarios con:");
console.log('  pnpm create-user -- correo@dominio.com "clave" "Nombre" student');
console.log('  pnpm create-user -- correo@dominio.com "clave" "Nombre" librarian');
```

Paso 9.9 · Pruebas de la semana

Configura Vitest. `server-only` lanza un error fuera de Next, así que en las pruebas lo reemplazamos por un módulo vacío:

▸ **CREAR · library_api/vitest.config.ts**

```
import { fileURLToPath } from "node:url";
import { defineConfig } from "vitest/config";

const root = (p: string) => fileURLToPath(new URL(p, import.meta.url));

export default defineConfig({
  resolve: {
    alias: {
      "@": root("./"),
      // `server-only` lanza error fuera de Next; en pruebas lo reemplazamos por un módulo vacío.
      "server-only": root("./tests/stubs/empty.ts"),
    },
  },
  test: { include: ["tests/**/*.test.ts"] },
});
```

▸ **CREAR · library_api/tests/stubs/empty.ts**

```
export {};
```

Dos archivos de pruebas: uno para los esquemas (incluye el caso del correo con espacios) y otro para el manejo de errores HTTP:

▸ **CREAR · library_api/tests/schemas.test.ts**

```
import { describe, expect, it } from "vitest";
import {
  bookQuerySchema,
  coverSchema,
  createLoanSchema,
  loginSchema,
  registerSchema,
} from "@/lib/schemas";

describe("loginSchema", () => {
  it("normaliza el correo", () => {
    expect(loginSchema.parse({ email: "  Ana@Mail.COM ", password: "x" }).email).toBe(
      "ana@mail.com",
    );
  });
  it("rechaza correos inválidos", () => {
    expect(loginSchema.safeParse({ email: "nope", password: "x" }).success).toBe(false);
  });
});

describe("registerSchema", () => {
  it("exige contraseña de 6+ caracteres (mínimo de Firebase)", () => {
    expect(
      registerSchema.safeParse({ email: "a@b.co", password: "12345", displayName: "Ana" }).success,
    ).toBe(false);
    expect(
      registerSchema.safeParse({ email: "a@b.co", password: "123456", displayName: "Ana" }).success,
    ).toBe(true);
  });
});

describe("createLoanSchema", () => {
  it("acepta uuid como clientRequestId", () => {
    expect(
      createLoanSchema.safeParse({
        bookId: "clean-code",
        clientRequestId: "3f2b8c1e-7d44-4b9a-9a1e-0c5d2f6a7b11",
      }).success,
    ).toBe(true);
  });
  it("rechaza ids con caracteres raros o muy cortos", () => {
    expect(createLoanSchema.safeParse({ bookId: "x", clientRequestId: "abc" }).success).toBe(false);
```

```
    expect(
      createLoanSchema.safeParse({ bookId: "x", clientRequestId: "../../etc/passwd" }).success,
    ).toBe(false);
  });
});

describe("coverSchema", () => {
  it("rechaza tipos no permitidos y base64 inválido", () => {
    const data = "A".repeat(200);
    expect(coverSchema.safeParse({ contentType: "image/gif", data }).success).toBe(false);
    expect(
      coverSchema.safeParse({ contentType: "image/jpeg", data: "no es base64!!" + data }).success,
    ).toBe(false);
    expect(coverSchema.safeParse({ contentType: "image/jpeg", data }).success).toBe(true);
  });
  it("rechaza imágenes por encima del límite", () => {
    expect(
      coverSchema.safeParse({ contentType: "image/png", data: "A".repeat(700_001) }).success,
    ).toBe(false);
  });
});

describe("bookQuerySchema", () => {
  it("valida updatedSince como fecha ISO", () => {
    expect(bookQuerySchema.safeParse({ updatedSince: "2026-10-12T10:00:00.000Z" }).success).toBe(
      true,
    );
    expect(bookQuerySchema.safeParse({ updatedSince: "ayer" }).success).toBe(false);
  });
});
```

▸ **CREAR · library_api/tests/http.test.ts**

```
import { describe, expect, it, vi } from "vitest";
import { z } from "zod";
import { ApiError } from "@/lib/errors";
import { parseBody, route } from "@/lib/http";

describe("route()", () => {
  it("traduce ApiError a { error: { code, message } } con su status", async () => {
    const handler = route(async () => {
      throw new ApiError(409, "NO_COPIES", "Sin ejemplares");
    });
    const res = await handler(new Request("http://x/api"), undefined);
    expect(res.status).toBe(409);
    expect(await res.json()).toMatchObject({
      error: { code: "NO_COPIES", message: "Sin ejemplares" },
    });
  });

  it("traduce errores de validación a 400", async () => {
    const handler = route(async (req) => {
      await parseBody(req, z.object({ n: z.number() }));
      return Response.json({ ok: true });
    });
    const res = await handler(
      new Request("http://x/api", { method: "POST", body: JSON.stringify({ n: "a" }) }),
      undefined,
    );
    expect(res.status).toBe(400);
    expect((await res.json()).error.code).toBe("VALIDATION_ERROR");
  });

  it("responde 400 si el cuerpo no es JSON", async () => {
    const handler = route(async (req) => Response.json(await parseBody(req, z.object({}))));
    const res = await handler(
      new Request("http://x/api", { method: "POST", body: "no-json" }),
      undefined,
    );
    expect(res.status).toBe(400);
  });

  it("oculta los detalles de errores inesperados", async () => {
    const spy = vi.spyOn(console, "error").mockImplementation(() => {});
    const handler = route(async () => {
      throw new Error("secreto interno de Firestore");
    });
    const res = await handler(new Request("http://x/api"), undefined);
```

```
    const text = await res.text();
    expect(res.status).toBe(500);
    expect(text).not.toContain("secreto");
    spy.mockRestore();
  });
});
```

> **Nota.** **Punto de control de la semana 9.** `pnpm typecheck` y `pnpm test` sin errores (verás 2 archivos de pruebas en verde).

3.3 Cómo probar con datos reales

```
pnpm seed                     # carga los 10 libros (necesita el .env)
pnpm dev                      # en otra terminal

curl -i http://localhost:3000/api/health
curl -s http://localhost:3000/api/books
curl -s 'http://localhost:3000/api/books?q=colera'
curl -s 'http://localhost:3000/api/books?category=Tecnolog%C3%ADa'
curl -s http://localhost:3000/api/books/rayuela
curl -i http://localhost:3000/api/books/no-existe        # 404
curl -i 'http://localhost:3000/api/books?updatedSince=ayer'   # 400: fecha inválida
```

También puedes usar **Bruno** o **Postman**: crea una colección con la variable `baseUrl`.

3.4 Evidencia y criterios

> **Evidencia para el docente.** Endpoints de catálogo funcionando con validación y errores uniformes; colección de pruebas (curl, Bruno o Postman) con respuestas válidas y de error; `pnpm test` en verde.

- ☐ Todas las entradas pasan por un esquema de Zod
- ☐ Todos los errores tienen la forma `{ error: { code, message } }`
- ☐ Un error inesperado devuelve `500` **sin** mostrar el detalle interno
- ☐ La API responde `404` y `400` donde corresponde

4. Semana 10 · Autenticación y autorización con Firebase

Del microcurrículo: Autenticación, autorización y protección de rutas. Manejo de sesión y tokens; secretos, datos personales y registro seguro.

4.1 Conceptos clave

- **Dos APIs de Firebase, dos usos.** El **Admin SDK** (cuenta de servicio) puede todo: crear usuarios, verificar tokens, asignar roles. La **API REST de Auth** (con la `apiKey` web) permite iniciar sesión con correo y contraseña. El servidor usa ambas; la app, ninguna.
- `idToken`**.** JWT firmado por Firebase que dura 1 hora. El servidor lo **verifica** en cada petición.
- `refreshToken`**.** Sirve para pedir un `idToken` nuevo (`/api/auth/refresh`).
- **Custom claims.** Datos extra firmados dentro del token. Guardamos `role: "student" | "librarian"`; así autorizar no cuesta una consulta a la base de datos.
- **Autorización ≠ autenticación.** Autenticar es saber quién eres; autorizar es decidir qué puedes hacer. Un estudiante autenticado sigue sin poder aprobar préstamos.

4.2 Paso a paso

Paso 10.1 · El servicio de autenticación

Verifica tokens, inicia sesión con la API REST de Firebase, renueva sesiones y registra usuarios. Tres detalles de seguridad:

- Un error de credenciales se traduce **siempre al mismo mensaje**, para no revelar si un correo existe.
- El registro público **siempre crea estudiantes**: el rol bibliotecario no se puede pedir desde el registro; se asigna con un script de administración.
- Se inicia sesión **después** de fijar los claims, para que el primer `idToken` ya incluya el rol.

▸ **CREAR · library_api/lib/services/auth.ts**

```
import "server-only";
import { adminAuth, db } from "../firebase";
import { getEnv } from "../env";
import { ApiError } from "../errors";
import type { AuthUser, Role, TokenSet } from "../types";

const IDENTITY = "https://identitytoolkit.googleapis.com/v1";
const SECURE_TOKEN = "https://securetoken.googleapis.com/v1";

/** 5 días: máximo razonable para la cookie de sesión web (Firebase admite hasta 14). */
export const SESSION_MAX_AGE_MS = 5 * 24 * 60 * 60 * 1000;

export interface AuthResult {
  tokens: TokenSet;
  user: AuthUser;
}

function toRole(value: unknown): Role {
  return value === "librarian" ? "librarian" : "student";
}

function toAuthUser(decoded: {
  uid: string;
  email?: string;
  name?: string;
```

```
  role?: unknown;
}): AuthUser {
  return {
    uid: decoded.uid,
    email: decoded.email ?? "",
    name: decoded.name ?? decoded.email ?? "Usuario",
    role: toRole(decoded.role),
  };
}

/** Verifica un idToken de Firebase (Authorization: Bearer ...) y devuelve el usuario con su rol. */
export async function verifyIdToken(idToken: string): Promise<AuthUser> {
  try {
    return toAuthUser(await adminAuth().verifyIdToken(idToken));
  } catch {
    throw new ApiError(401, "UNAUTHENTICATED", "Sesión inválida o expirada");
  }
}

/** Verifica la cookie de sesión de la web. */
export async function verifySessionCookie(cookie: string): Promise<AuthUser> {
  try {
    return toAuthUser(await adminAuth().verifySessionCookie(cookie));
  } catch {
    throw new ApiError(401, "UNAUTHENTICATED", "Sesión inválida o expirada");
  }
}

export async function createSessionCookie(idToken: string): Promise<string> {
  return adminAuth().createSessionCookie(idToken, { expiresIn: SESSION_MAX_AGE_MS });
}

async function firebaseRest<T>(url: string, init: RequestInit): Promise<T> {
  const res = await fetch(url, { ...init, cache: "no-store" });
  const body = (await res.json().catch(() => ({}))) as { error?: { message?: string } } & T;
  if (!res.ok) {
    const code = body.error?.message ?? "UNKNOWN";
    if (
      code.startsWith("INVALID_LOGIN_CREDENTIALS") ||
      code.startsWith("INVALID_PASSWORD") ||
      code.startsWith("EMAIL_NOT_FOUND") ||
      code.startsWith("INVALID_REFRESH_TOKEN") ||
      code.startsWith("TOKEN_EXPIRED")
    ) {
      throw new ApiError(401, "INVALID_CREDENTIALS", "Correo o contraseña incorrectos");
    }
    if (code.startsWith("USER_DISABLED")) {
      throw new ApiError(403, "FORBIDDEN", "Tu cuenta está deshabilitada");
    }
    if (code.startsWith("TOO_MANY_ATTEMPTS")) {
      throw new ApiError(429, "RATE_LIMITED", "Demasiados intentos, espera un momento");
    }
    throw new ApiError(502, "INTERNAL_ERROR", "Error al contactar el servicio de autenticación");
  }
  return body;
}

/** Login con correo y contraseña usando la API REST de Firebase Auth. */
export async function login(email: string, password: string): Promise<AuthResult> {
  const key = getEnv().FIREBASE_WEB_API_KEY;
  const data = await firebaseRest<{ idToken: string; refreshToken: string; expiresIn: string }>(
    `${IDENTITY}/accounts:signInWithPassword?key=${key}`,
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ email, password, returnSecureToken: true }),
    },
  );
  return {
    tokens: {
      idToken: data.idToken,
      refreshToken: data.refreshToken,
      expiresIn: Number(data.expiresIn),
    },
    user: await verifyIdToken(data.idToken),
  };
}
```

```
/** Intercambia el refreshToken por un idToken nuevo (la app lo usa al recibir 401). */
export async function refresh(refreshToken: string): Promise<AuthResult> {
  const key = getEnv().FIREBASE_WEB_API_KEY;
  const data = await firebaseRest<{ id_token: string; refresh_token: string; expires_in: string }>(
    `${SECURE_TOKEN}/token?key=${key}`,
    {
      method: "POST",
      headers: { "Content-Type": "application/x-www-form-urlencoded" },
      body: new URLSearchParams({ grant_type: "refresh_token", refresh_token: refreshToken }),
    },
  );
  return {
    tokens: {
      idToken: data.id_token,
      refreshToken: data.refresh_token,
      expiresIn: Number(data.expires_in),
    },
    user: await verifyIdToken(data.id_token),
  };
}

/** Registro público: siempre crea un estudiante. El rol bibliotecario solo se asigna con scripts/set-role.ts. */
export async function register(
  email: string,
  password: string,
  displayName: string,
): Promise<AuthResult> {
  const auth = adminAuth();
  try {
    const created = await auth.createUser({ email, password, displayName });
    await auth.setCustomUserClaims(created.uid, { role: "student" });
    await db().collection("users").doc(created.uid).set({
      email,
      displayName,
      role: "student",
      createdAt: new Date().toISOString(),
    });
  } catch (err) {
    const code = (err as { code?: string }).code;
    if (code === "auth/email-already-exists") {
      throw new ApiError(409, "EMAIL_IN_USE", "Ya existe una cuenta con ese correo");
    }
    if (code === "auth/invalid-password" || code === "auth/invalid-email") {
      throw new ApiError(400, "VALIDATION_ERROR", "Correo o contraseña inválidos");
    }
    throw err;
  }
  // Se inicia sesión DESPUÉS de fijar los claims para que el idToken ya incluya el rol.
  return login(email, password);
}
```

Paso 10.2 · Autenticar y autorizar en cada ruta

Dos funciones pequeñas hacen toda la autorización de la API: `requireUser` exige un Bearer token válido y `assertRole` exige un rol.

▸ **CREAR · library_api/lib/guard.ts**

```
import "server-only";
import { ApiError } from "./errors";
import { verifyIdToken } from "./services/auth";
import type { AuthUser, Role } from "./types";

/** Autenticación de la API móvil: Authorization: Bearer <idToken de Firebase>. */
export async function requireUser(request: Request): Promise<AuthUser> {
  const header = request.headers.get("authorization") ?? "";
  const [scheme, token] = header.split(" ");
  if (scheme?.toLowerCase() !== "bearer" || !token) {
    throw new ApiError(401, "UNAUTHENTICATED", "Falta el token de autenticación");
  }
  return verifyIdToken(token);
}

/** Autorización por rol (el rol viaja como custom claim dentro del idToken). */
export function assertRole(user: AuthUser, role: Role): void {
  if (user.role !== role) {
    throw new ApiError(403, "FORBIDDEN", "No tienes permisos para esta acción");
```

```
  }
}
```

Paso 10.3 · Las rutas de autenticación

Cuatro endpoints delgados: validan con Zod, llaman al servicio y responden.

▸ **CREAR · library_api/app/api/auth/login/route.ts**

```ts
import { parseBody, route } from "@/lib/http";
import { loginSchema } from "@/lib/schemas";
import { login } from "@/lib/services/auth";

export const POST = route(async (request) => {
  const { email, password } = await parseBody(request, loginSchema);
  return Response.json(await login(email, password));
});
```

▸ **CREAR · library_api/app/api/auth/register/route.ts**

```ts
import { parseBody, route } from "@/lib/http";
import { registerSchema } from "@/lib/schemas";
import { register } from "@/lib/services/auth";

export const POST = route(async (request) => {
  const { email, password, displayName } = await parseBody(request, registerSchema);
  return Response.json(await register(email, password, displayName), { status: 201 });
});
```

▸ **CREAR · library_api/app/api/auth/refresh/route.ts**

```ts
import { parseBody, route } from "@/lib/http";
import { refreshSchema } from "@/lib/schemas";
import { refresh } from "@/lib/services/auth";

export const POST = route(async (request) => {
  const { refreshToken } = await parseBody(request, refreshSchema);
  return Response.json(await refresh(refreshToken));
});
```

▸ **CREAR · library_api/app/api/auth/me/route.ts**

```ts
import { requireUser } from "@/lib/guard";
import { route } from "@/lib/http";

export const GET = route(async (request) => {
  return Response.json({ user: await requireUser(request) });
});
```

Paso 10.4 · Proteger el catálogo

Ahora sí: reemplaza las dos rutas de la semana 9 por estas versiones. La **única diferencia** es la línea `await requireUser(request)` al inicio (y su `import`).

▸ **CREAR · library_api/app/api/books/route.ts**

```ts
import { listBooks, listCategories } from "@/lib/data/books";
import { requireUser } from "@/lib/guard";
import { parseQuery, route } from "@/lib/http";
import { bookQuerySchema } from "@/lib/schemas";

/**
 * GET /api/books?q=&category=&updatedSince=
 * `updatedSince` (ISO) devuelve solo los libros modificados desde esa fecha:
 * es el cursor de la sincronización incremental de la app (semana 11).
 * `serverTime` se toma ANTES de consultar para no perder cambios concurrentes.
 */
export const GET = route(async (request) => {
  await requireUser(request);
  const filter = parseQuery(request, bookQuerySchema);
  const serverTime = new Date().toISOString();
  const [books, categories] = await Promise.all([listBooks(filter), listCategories()]);
  return Response.json({ books, categories, serverTime });
});
```

```
});
```

▸ **CREAR · library_api/app/api/books/[id]/route.ts**

```
import { getBook } from "@/lib/data/books";
import { requireUser } from "@/lib/guard";
import { route } from "@/lib/http";

export const GET = route(async (request, ctx: RouteContext<"/api/books/[id]">) => {
  await requireUser(request);
  const { id } = await ctx.params;
  return Response.json({ book: await getBook(id) });
});
```

Paso 10.5 · Scripts de administración

Para crear usuarios con un rol concreto (el primer bibliotecario real) y para cambiar el rol de un usuario existente:

▸ **CREAR · library_api/scripts/create-user.mts**

```
/**
 * Crea (o actualiza) un usuario con un rol concreto. Sirve para dar de alta al primer bibliotecario real.
 *   pnpm create-user -- correo@dominio.com "Contraseña" "Nombre Apellido" librarian
 * Si el correo ya existe, se le asigna la contraseña, el nombre y el rol indicados.
 * Ojo: la contraseña queda en el historial de tu terminal; cámbiala luego desde la consola de Firebase si es sensible.
 */
import { cert, initializeApp } from "firebase-admin/app";
import { getAuth } from "firebase-admin/auth";
import { getFirestore } from "firebase-admin/firestore";

const [email, password, displayName, role] = process.argv.slice(2).filter((a) => a !== "--");
if (!email || !password || !displayName || (role !== "librarian" && role !== "student")) {
  console.error('Uso: pnpm create-user -- <correo> <contraseña> "<nombre>" <librarian|student>');
  process.exit(1);
}
if (password.length < 6) {
  console.error("Firebase exige contraseñas de al menos 6 caracteres.");
  process.exit(1);
}

initializeApp({
  credential: cert({
    projectId: process.env.FIREBASE_PROJECT_ID!,
    clientEmail: process.env.FIREBASE_CLIENT_EMAIL!,
    privateKey: (process.env.FIREBASE_PRIVATE_KEY ?? "").replace(/\\n/g, "\n"),
  }),
});
const auth = getAuth();

let uid: string;
try {
  uid = (await auth.getUserByEmail(email)).uid;
  await auth.updateUser(uid, { password, displayName });
  console.log(`= usuario existente actualizado: ${email}`);
} catch (err) {
  if ((err as { code?: string }).code !== "auth/user-not-found") throw err;
  uid = (await auth.createUser({ email, password, displayName })).uid;
  console.log(`+ usuario creado: ${email}`);
}

await auth.setCustomUserClaims(uid, { role });
await auth.revokeRefreshTokens(uid);
await getFirestore()
  .collection("users")
  .doc(uid)
  .set({ email, displayName, role, createdAt: new Date().toISOString() }, { merge: true });

// Comprobación real: iniciar sesión y leer el rol que viaja dentro del token.
const res = await fetch(
  `https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key=${process.env.FIREBASE_WEB_API_KEY}`,
  {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password, returnSecureToken: true }),
  },
```

```
);
const body = (await res.json()) as { idToken?: string; error?: { message?: string } };
if (!body.idToken) {
  console.error("✗ No se pudo iniciar sesión:", body.error?.message);
  process.exit(1);
}
const claims = await auth.verifyIdToken(body.idToken);
console.log(`✓ Inicio de sesión correcto. Rol en el token: ${claims.role}`);
```

▸ **CREAR · library_api/scripts/set-role.mts**

```
/**
 * Asigna el rol de un usuario existente.
 *   pnpm set-role -- correo@dominio.com librarian
 *   pnpm set-role -- correo@dominio.com student
 * El usuario debe cerrar sesión y volver a entrar (o refrescar el token) para ver el cambio.
 */
import { cert, initializeApp } from "firebase-admin/app";
import { getAuth } from "firebase-admin/auth";
import { getFirestore } from "firebase-admin/firestore";

const [email, role] = process.argv.slice(2).filter((a) => a !== "--");
if (!email || (role !== "librarian" && role !== "student")) {
  console.error("Uso: pnpm set-role -- <correo> <librarian|student>");
  process.exit(1);
}

initializeApp({
  credential: cert({
    projectId: process.env.FIREBASE_PROJECT_ID!,
    clientEmail: process.env.FIREBASE_CLIENT_EMAIL!,
    privateKey: (process.env.FIREBASE_PRIVATE_KEY ?? "").replace(/\\n/g, "\n"),
  }),
});

const user = await getAuth().getUserByEmail(email);
await getAuth().setCustomUserClaims(user.uid, { role });
await getAuth().revokeRefreshTokens(user.uid);
await getFirestore().collection("users").doc(user.uid).set({ role }, { merge: true });
console.log(`${email} ahora es ${role}`);
```

`revokeRefreshTokens` obliga a renovar la sesión, de modo que el cambio de rol se aplique pronto y no solo cuando expire el token anterior.

Paso 10.6 · Pruebas de la semana

La prueba del guard simula Firebase y comprueba: `401` sin cabecera, solo esquema `Bearer`, usuario verificado y `403` por rol.

▸ **CREAR · library_api/tests/guard.test.ts**

```
import { beforeEach, describe, expect, it, vi } from "vitest";
import { ApiError } from "@/lib/errors";

const verifyIdToken = vi.fn();
vi.mock("@/lib/services/auth", () => ({ verifyIdToken: (t: string) => verifyIdToken(t) }));

import { assertRole, requireUser } from "@/lib/guard";

const req = (authorization?: string) =>
  new Request("http://x/api", { headers: authorization ? { authorization } : {} });

beforeEach(() => verifyIdToken.mockReset());

describe("requireUser", () => {
  it("rechaza peticiones sin cabecera Authorization", async () => {
    await expect(requireUser(req())).rejects.toMatchObject({
      status: 401,
      code: "UNAUTHENTICATED",
    });
  });

  it("rechaza esquemas distintos de Bearer", async () => {
    await expect(requireUser(req("Basic abc"))).rejects.toMatchObject({ status: 401 });
    expect(verifyIdToken).not.toHaveBeenCalled();
```

```
  });

  it("devuelve el usuario verificado", async () => {
    const user = { uid: "u1", email: "a@b.co", name: "Ana", role: "student" };
    verifyIdToken.mockResolvedValue(user);
    await expect(requireUser(req("Bearer tok"))).resolves.toEqual(user);
    expect(verifyIdToken).toHaveBeenCalledWith("tok");
  });
});

describe("assertRole", () => {
  const student = { uid: "u1", email: "a@b.co", name: "Ana", role: "student" as const };
  it("permite el rol correcto", () => {
    expect(() => assertRole(student, "student")).not.toThrow();
  });
  it("lanza 403 si el rol no coincide", () => {
    try {
      assertRole(student, "librarian");
      throw new Error("debía fallar");
    } catch (e) {
      expect(e).toBeInstanceOf(ApiError);
      expect((e as ApiError).status).toBe(403);
    }
  });
});
```

Nota. **Punto de control de la semana 10.** `pnpm typecheck` y `pnpm test` en verde (3 archivos de pruebas).

4.3 Matriz de autorización

Endpoint	Sin sesión	Estudiante	Bibliotecario
`GET /api/books`, `/api/books/:id`	401	✓	✓
`POST /api/loans`	401	✓	403
`GET /api/loans`	401	Solo los suyos	Todos
`GET /api/loans/:id`	401	Solo si es suyo; si no, 404	✓
`PATCH /api/loans/:id`	401	403	✓
`PUT /api/books/:id/cover`	401	403	✓

Nota. Un estudiante que pide por id un préstamo **ajeno** recibe `404`, no `403`: un `403` confirmaría que ese préstamo existe.

4.4 Cómo probar

```
# Sin token
curl -i http://localhost:3000/api/books                       # 401

# Crea TUS usuarios de prueba (con tus propias contraseñas; no quedan escritas en ningún archivo)
pnpm create-user -- estudiante@tucorreo.com "TuClaveEstudiante" "Tu Estudiante" student
pnpm create-user -- bibliotecario@tucorreo.com "TuClaveBibliotecario" "Tu Bibliotecario" librarian

# También puedes registrar un estudiante desde la API (el registro público siempre crea estudiantes)
curl -s -X POST http://localhost:3000/api/auth/register -H 'Content-Type: application/json' \
  -d '{"email":"ana@correo.co","password":"secreta1","displayName":"Ana"}'

# Iniciar sesión y guardar el token
TOKEN=$(curl -s -X POST http://localhost:3000/api/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"estudiante@tucorreo.com","password":"TuClaveEstudiante"}' \
  | node -pe "JSON.parse(require('fs').readFileSync(0,'utf8')).tokens.idToken")

curl -s -H "Authorization: Bearer $TOKEN" http://localhost:3000/api/auth/me
curl -s -H "Authorization: Bearer $TOKEN" http://localhost:3000/api/books/rayuela
```

4.5 Amenazas y controles

Amenaza	Control
Secretos de administrador filtrados	Solo en variables de entorno; `*-firebase-adminsdk-*.json` y `.env*` ignorados por git
Escalada de privilegios al registrarse	El registro público siempre crea `student`
Un estudiante invoca endpoints de bibliotecario	`assertRole` en cada ruta (se prueba con `403`)
Enumeración de correos	Mensaje único para credenciales inválidas; `404` en vez de `403` para recursos ajenos
Entradas maliciosas	Validación con Zod en todas las rutas

4.6 Evidencia y criterios

> **Evidencia para el docente.** Registro, login, refresh y `me` funcionando; roles por custom claims; matriz de autorización comprobada con peticiones manuales; tabla de amenazas del proyecto.

- ☐ Sin token, toda ruta privada responde `401`
- ☐ El registro no permite elegir rol
- ☐ Ningún secreto está en el repositorio (revisa con `git status` y `git log`)

5. Semana 11 · Préstamos, sincronización y conflictos en el servidor

Del microcurrículo: Sincronización local-remota: caché, cola de cambios, conflictos y consistencia eventual. La app implementará el lado del teléfono (Guía 2); la API debe ofrecer tres garantías para que sea **segura**:

Garantía	Qué problema resuelve	Cómo se logra
Cambios incrementales	No descargar todo el catálogo cada vez	`updatedSince` + `serverTime` como cursor (ya lo hiciste)
Idempotencia	Reintentar tras perder la respuesta no debe duplicar préstamos	El id del documento se deriva de `{uid}_{clientRequestId}`
Consistencia de ejemplares	Dos estudiantes no pueden llevarse el último ejemplar a la vez	Transacciones de Firestore

5.1 Paso a paso

Paso 11.1 · Reglas de dominio puras

Las reglas de negocio son **funciones puras** (sin Firebase ni red): se prueban en milisegundos y se reutilizan igual desde la API y desde la web. Un préstamo está "activo" mientras sea `requested` o `approved`. Los errores `409` de aquí son los **conflictos** que la app móvil explicará al usuario cuando una solicitud hecha sin conexión llegue tarde.

▸ **CREAR · library_api/lib/domain/loan-rules.ts**

```
import { ApiError } from "../errors";
import type { Book, Loan, LoanAction } from "../types";

/** Un préstamo "activo" bloquea una nueva solicitud del mismo libro por el mismo usuario. */
const ACTIVE_STATUSES: Loan["status"][] = ["requested", "approved"];

export const LOAN_DAYS = 14;

export function isActive(loan: Pick<Loan, "status">): boolean {
  return ACTIVE_STATUSES.includes(loan.status);
}

/**
 * Valida que el estudiante pueda solicitar el libro.
 * Estos errores 409 son los "conflictos" que la app debe saber resolver cuando
 * una solicitud creada sin conexión llega tarde al servidor (semana 11).
 */
export function assertCanRequest(
  book: Pick<Book, "id" | "availableCopies">,
  userLoans: Pick<Loan, "bookId" | "status">[],
): void {
  if (userLoans.some((l) => l.bookId === book.id && isActive(l))) {
    throw new ApiError(
      409,
      "ALREADY_ACTIVE",
      "Ya tienes una solicitud o préstamo activo de este libro",
    );
  }
  if (book.availableCopies <= 0) {
    throw new ApiError(409, "NO_COPIES", "No hay ejemplares disponibles");
  }
}

export interface Transition {
  loan: Pick<Loan, "status" | "resolvedAt" | "dueDate" | "returnedAt" | "updatedAt">;
  /** Cambio en ejemplares disponibles del libro: -1 al prestar, +1 al devolver. */
  copiesDelta: -1 | 0 | 1;
```

```
}

/** Máquina de estados: requested → approved → returned, o requested → rejected. */
export function applyAction(
  loan: Pick<Loan, "status" | "dueDate">,
  book: Pick<Book, "availableCopies" | "totalCopies">,
  action: LoanAction,
  now: Date,
): Transition {
  const iso = now.toISOString();

  if (action === "approve") {
    if (loan.status !== "requested") {
      throw new ApiError(409, "INVALID_STATE", "Solo se puede aprobar una solicitud pendiente");
    }
    if (book.availableCopies <= 0) {
      throw new ApiError(409, "NO_COPIES", "No hay ejemplares disponibles para prestar");
    }
    const due = new Date(now.getTime() + LOAN_DAYS * 24 * 60 * 60 * 1000);
    return {
      loan: {
        status: "approved",
        resolvedAt: iso,
        dueDate: due.toISOString(),
        returnedAt: null,
        updatedAt: iso,
      },
      copiesDelta: -1,
    };
  }

  if (action === "reject") {
    if (loan.status !== "requested") {
      throw new ApiError(409, "INVALID_STATE", "Solo se puede rechazar una solicitud pendiente");
    }
    return {
      loan: {
        status: "rejected",
        resolvedAt: iso,
        dueDate: loan.dueDate,
        returnedAt: null,
        updatedAt: iso,
      },
      copiesDelta: 0,
    };
  }

  if (loan.status !== "approved") {
    throw new ApiError(409, "INVALID_STATE", "Solo se puede devolver un préstamo aprobado");
  }
  return {
    loan: {
      status: "returned",
      resolvedAt: iso,
      dueDate: loan.dueDate,
      returnedAt: iso,
      updatedAt: iso,
    },
    copiesDelta: book.availableCopies < book.totalCopies ? 1 : 0,
  };
}
```

Paso 11.2 · Acceso a los préstamos en Firestore

Lee con calma las dos funciones principales:

- `requestLoan` busca **primero** el documento con id `{uid}_{clientRequestId}`. Si existe, lo devuelve (`created: false`): por eso es seguro reintentar. Todo ocurre en una **transacción**.
- **Solicitar no reserva ejemplares**; solo prestar los descuenta. Dos estudiantes pueden solicitar el último ejemplar: el bibliotecario aprueba uno y la segunda aprobación falla con `NO_COPIES`.
- `transitionLoan` aplica la máquina de estados y descuenta o devuelve el ejemplar de forma atómica.

▸ **CREAR · library_api/lib/data/loans.ts**

```
import "server-only";
import { db } from "../firebase";
import { ApiError } from "../errors";
import { applyAction, assertCanRequest } from "../domain/loan-rules";
import type { AuthUser, Book, Loan, LoanAction, LoanStatus } from "../types";
import { BOOKS } from "./books";

const LOANS = "loans";

const byNewest = (a: Loan, b: Loan) => b.requestedAt.localeCompare(a.requestedAt);

function toLoan(id: string, data: Omit<Loan, "id">): Loan {
  return { id, ...data };
}

/**
 * Crea una solicitud de préstamo de forma idempotente.
 * El id del documento se deriva de (uid, clientRequestId): si la app reenvía la misma
 * solicitud tras perder conexión, se devuelve la ya creada en lugar de duplicarla.
 */
export async function requestLoan(
  user: AuthUser,
  bookId: string,
  clientRequestId: string,
): Promise<{ loan: Loan; created: boolean }> {
  const firestore = db();
  const loanRef = firestore.collection(LOANS).doc(`${user.uid}_${clientRequestId}`);
  const bookRef = firestore.collection(BOOKS).doc(bookId);

  return firestore.runTransaction(async (tx) => {
    const existing = await tx.get(loanRef);
    if (existing.exists) {
      return { loan: toLoan(existing.id, existing.data() as Omit<Loan, "id">), created: false };
    }

    const [bookSnap, mine] = await Promise.all([
      tx.get(bookRef),
      tx.get(firestore.collection(LOANS).where("userId", "==", user.uid)),
    ]);
    if (!bookSnap.exists) throw new ApiError(404, "NOT_FOUND", "Libro no encontrado");
    const book = bookSnap.data() as Book;

    assertCanRequest(
      { id: bookSnap.id, availableCopies: book.availableCopies },
      mine.docs.map((d) => d.data() as Loan),
    );

    const now = new Date().toISOString();
    const data: Omit<Loan, "id"> = {
      bookId,
      bookTitle: book.title,
      userId: user.uid,
      userEmail: user.email,
      userName: user.name,
      status: "requested",
      clientRequestId,
      requestedAt: now,
      resolvedAt: null,
      dueDate: null,
      returnedAt: null,
      updatedAt: now,
    };
    tx.set(loanRef, data);
    return { loan: toLoan(loanRef.id, data), created: true };
  });
}

/** Estudiante: solo sus préstamos. Bibliotecario: todos (con filtro opcional por estado). */
export async function listLoans(user: AuthUser, status?: LoanStatus): Promise<Loan[]> {
  let query: FirebaseFirestore.Query = db().collection(LOANS);
  if (user.role === "student") query = query.where("userId", "==", user.uid);
  else if (status) query = query.where("status", "==", status);

  const snap = await query.get();
  let loans = snap.docs.map((d) => toLoan(d.id, d.data() as Omit<Loan, "id">));
  if (user.role === "student" && status) loans = loans.filter((l) => l.status === status);
```

```
  return loans.sort(byNewest);
}

export async function getLoan(user: AuthUser, id: string): Promise<Loan> {
  const snap = await db().collection(LOANS).doc(id).get();
  if (!snap.exists) throw new ApiError(404, "NOT_FOUND", "Préstamo no encontrado");
  const loan = toLoan(snap.id, snap.data() as Omit<Loan, "id">);
  // Un estudiante no debe poder distinguir préstamos ajenos de los inexistentes.
  if (user.role === "student" && loan.userId !== user.uid) {
    throw new ApiError(404, "NOT_FOUND", "Préstamo no encontrado");
  }
  return loan;
}

/** Aprobar / rechazar / devolver: solo bibliotecario (se valida en la ruta). */
export async function transitionLoan(id: string, action: LoanAction): Promise<Loan> {
  const firestore = db();
  const loanRef = firestore.collection(LOANS).doc(id);

  return firestore.runTransaction(async (tx) => {
    const loanSnap = await tx.get(loanRef);
    if (!loanSnap.exists) throw new ApiError(404, "NOT_FOUND", "Préstamo no encontrado");
    const loan = toLoan(loanSnap.id, loanSnap.data() as Omit<Loan, "id">);

    const bookRef = firestore.collection(BOOKS).doc(loan.bookId);
    const bookSnap = await tx.get(bookRef);
    if (!bookSnap.exists) throw new ApiError(404, "NOT_FOUND", "Libro no encontrado");
    const book = bookSnap.data() as Book;

    const now = new Date();
    const result = applyAction(loan, book, action, now);

    tx.update(loanRef, result.loan);
    if (result.copiesDelta !== 0) {
      tx.update(bookRef, {
        availableCopies: book.availableCopies + result.copiesDelta,
        updatedAt: now.toISOString(),
      });
    }
    return { ...loan, ...result.loan };
  });
}
```

Paso 11.3 · Las rutas de préstamos

`POST /api/loans` es idempotente: responde `201` si creó el préstamo y `200` si ya existía (reintento de la cola offline).

▸ **CREAR · library_api/app/api/loans/route.ts**

```
import { listLoans, requestLoan } from "@/lib/data/loans";
import { assertRole, requireUser } from "@/lib/guard";
import { parseBody, parseQuery, route } from "@/lib/http";
import { createLoanSchema, loanQuerySchema } from "@/lib/schemas";

/** Estudiante: sus préstamos. Bibliotecario: todos. */
export const GET = route(async (request) => {
  const user = await requireUser(request);
  const { status } = parseQuery(request, loanQuerySchema);
  const serverTime = new Date().toISOString();
  return Response.json({ loans: await listLoans(user, status), serverTime });
});

/**
 * POST /api/loans  { bookId, clientRequestId }
 * Idempotente por clientRequestId: 201 si se creó, 200 si ya existía (reintento de la cola offline).
 * 409 ALREADY_ACTIVE / NO_COPIES son los conflictos que la app resuelve "gana el servidor".
 */
export const POST = route(async (request) => {
  const user = await requireUser(request);
  assertRole(user, "student");
  const { bookId, clientRequestId } = await parseBody(request, createLoanSchema);
  const { loan, created } = await requestLoan(user, bookId, clientRequestId);
  return Response.json({ loan }, { status: created ? 201 : 200 });
});
```

▸ **CREAR · library_api/app/api/loans/[id]/route.ts**

```
import { getLoan, transitionLoan } from "@/lib/data/loans";
import { assertRole, requireUser } from "@/lib/guard";
import { parseBody, route } from "@/lib/http";
import { loanActionSchema } from "@/lib/schemas";

export const GET = route(async (request, ctx: RouteContext<"/api/loans/[id]">) => {
  const user = await requireUser(request);
  const { id } = await ctx.params;
  return Response.json({ loan: await getLoan(user, id) });
});

/** PATCH /api/loans/:id  { action: "approve" | "reject" | "return" } — solo bibliotecario. */
export const PATCH = route(async (request, ctx: RouteContext<"/api/loans/[id]">) => {
  const user = await requireUser(request);
  assertRole(user, "librarian");
  const { id } = await ctx.params;
  const { action } = await parseBody(request, loanActionSchema);
  return Response.json({ loan: await transitionLoan(id, action) });
});
```

Paso 11.4 · Pruebas de las reglas

▸ **CREAR · library_api/tests/loan-rules.test.ts**

```
import { describe, expect, it } from "vitest";
import { applyAction, assertCanRequest, LOAN_DAYS } from "@/lib/domain/loan-rules";
import { ApiError } from "@/lib/errors";

const now = new Date("2026-10-12T10:00:00.000Z");

function codeOf(fn: () => unknown): string | undefined {
  try {
    fn();
  } catch (e) {
    return (e as ApiError).code;
  }
}

describe("assertCanRequest", () => {
  it("permite solicitar un libro disponible sin préstamos activos", () => {
    expect(() => assertCanRequest({ id: "b1", availableCopies: 1 }, [])).not.toThrow();
  });

  it("permite volver a pedir un libro ya devuelto o rechazado", () => {
    const loans = [
      { bookId: "b1", status: "returned" as const },
      { bookId: "b1", status: "rejected" as const },
    ];
    expect(() => assertCanRequest({ id: "b1", availableCopies: 1 }, loans)).not.toThrow();
  });

  it("rechaza con ALREADY_ACTIVE si ya hay solicitud o préstamo del mismo libro", () => {
    for (const status of ["requested", "approved"] as const) {
      expect(
        codeOf(() =>
          assertCanRequest({ id: "b1", availableCopies: 2 }, [{ bookId: "b1", status }]),
        ),
      ).toBe("ALREADY_ACTIVE");
    }
  });

  it("no confunde préstamos activos de otro libro", () => {
    expect(() =>
      assertCanRequest({ id: "b1", availableCopies: 1 }, [{ bookId: "b2", status: "approved" }]),
    ).not.toThrow();
  });

  it("rechaza con NO_COPIES si no quedan ejemplares", () => {
    expect(codeOf(() => assertCanRequest({ id: "b1", availableCopies: 0 }, []))).toBe("NO_COPIES");
  });
});

describe("applyAction", () => {
  const book = { availableCopies: 2, totalCopies: 3 };
```

```
  it("approve: fija fecha de entrega a LOAN_DAYS y descuenta un ejemplar", () => {
    const t = applyAction({ status: "requested", dueDate: null }, book, "approve", now);
    expect(t.loan.status).toBe("approved");
    expect(t.copiesDelta).toBe(-1);
    const due = new Date(t.loan.dueDate!);
    expect((due.getTime() - now.getTime()) / 86_400_000).toBe(LOAN_DAYS);
  });

  it("approve: falla sin ejemplares disponibles", () => {
    const code = codeOf(() =>
      applyAction(
        { status: "requested", dueDate: null },
        { availableCopies: 0, totalCopies: 3 },
        "approve",
        now,
      ),
    );
    expect(code).toBe("NO_COPIES");
  });

  it("approve: solo desde requested", () => {
    for (const status of ["approved", "rejected", "returned"] as const) {
      expect(codeOf(() => applyAction({ status, dueDate: null }, book, "approve", now))).toBe(
        "INVALID_STATE",
      );
    }
  });

  it("reject: no cambia ejemplares", () => {
    const t = applyAction({ status: "requested", dueDate: null }, book, "reject", now);
    expect(t.loan.status).toBe("rejected");
    expect(t.copiesDelta).toBe(0);
  });

  it("return: devuelve el ejemplar y conserva la fecha de entrega", () => {
    const due = "2026-10-26T10:00:00.000Z";
    const t = applyAction({ status: "approved", dueDate: due }, book, "return", now);
    expect(t.loan.status).toBe("returned");
    expect(t.loan.dueDate).toBe(due);
    expect(t.copiesDelta).toBe(1);
  });

  it("return: nunca supera el total de ejemplares", () => {
    const t = applyAction(
      { status: "approved", dueDate: null },
      { availableCopies: 3, totalCopies: 3 },
      "return",
      now,
    );
    expect(t.copiesDelta).toBe(0);
  });

  it("return: solo desde approved", () => {
    expect(
      codeOf(() => applyAction({ status: "requested", dueDate: null }, book, "return", now)),
    ).toBe("INVALID_STATE");
  });
});
```

> **Nota. Punto de control de la semana 11.** `pnpm typecheck` y `pnpm test` en verde (4 archivos de pruebas).

5.2 Cómo probar

Con `TOKEN` de tu estudiante y `BIB` de tu bibliotecario (los que creaste en 4.4; inicia sesión con el bibliotecario igual que con el estudiante):

```
# Solicitar (201) y repetir con el MISMO clientRequestId (200, mismo id: idempotencia)
curl -s -X POST http://localhost:3000/api/loans -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"bookId":"clean-code","clientRequestId":"prueba-0001-abcdef"}'
curl -s -X POST http://localhost:3000/api/loans -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"bookId":"clean-code","clientRequestId":"prueba-0001-abcdef"}'
```

```
# Con OTRO clientRequestId para el mismo libro: 409 ALREADY_ACTIVE
curl -s -X POST http://localhost:3000/api/loans -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"bookId":"clean-code","clientRequestId":"prueba-0002-abcdef"}'

# El bibliotecario ve y aprueba (copia el id del préstamo)
curl -s -H "Authorization: Bearer $BIB" http://localhost:3000/api/loans
curl -s -X PATCH http://localhost:3000/api/loans/ID_DEL_PRESTAMO -H "Authorization: Bearer $BIB" \
  -H 'Content-Type: application/json' -d '{"action":"approve"}'

# El estudiante NO puede aprobar: 403
curl -i -X PATCH http://localhost:3000/api/loans/ID_DEL_PRESTAMO -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"action":"approve"}'
```

Para dejar los datos como al inicio: `pnpm reset-demo` (lo creas en el capítulo 8).

5.3 Evidencia y criterios

Evidencia para el docente. Reglas de préstamo con pruebas unitarias; demostración de idempotencia (misma solicitud dos veces → un solo préstamo) y de conflicto (`NO_COPIES`); explicación de por qué se eligió "gana el servidor".

- ☐ Reenviar la misma solicitud devuelve `200` con el mismo id
- ☐ Una segunda solicitud activa del mismo libro responde `409 ALREADY_ACTIVE`
- ☐ Aprobar sin ejemplares responde `409 NO_COPIES`
- ☐ `GET /api/books?updatedSince=<fecha futura>` devuelve una lista vacía

Nota. **Limitaciones conocidas, a propósito:** los listados se filtran y ordenan en memoria porque el volumen es pequeño (evita crear índices compuestos en Firestore). Con miles de registros habría que paginar y crear índices.

6. Semana 12 · Portadas: el servicio del dispositivo, visto desde el servidor

Del microcurrículo: Integración de un servicio del dispositivo con la API: imágenes… Permisos, validación, manejo de fallos y trazabilidad. La app toma la foto; la API **valida y guarda**.

6.1 Paso a paso

Paso 12.1 · No confiar en el tipo de archivo declarado

El cliente dice "esto es un PNG", pero no hay que creerle. Se comprueban los primeros bytes del archivo (magic bytes): un HTML renombrado como `.png` no pasa.

▸ **CREAR · library_api/lib/image.ts**

```
export type ImageType = "image/jpeg" | "image/png" | "image/webp";

/**
 * Detecta el tipo real de una imagen por sus "magic bytes".
 * No basta con confiar en el Content-Type que declara el cliente (MASVS / OWASP: validar entradas).
 */
export function sniffImageType(bytes: Uint8Array): ImageType | null {
  if (bytes.length >= 3 && bytes[0] === 0xff && bytes[1] === 0xd8 && bytes[2] === 0xff) {
    return "image/jpeg";
  }
  if (
    bytes.length >= 8 &&
    bytes[0] === 0x89 &&
    bytes[1] === 0x50 &&
    bytes[2] === 0x4e &&
    bytes[3] === 0x47
  ) {
    return "image/png";
  }
  if (
    bytes.length >= 12 &&
    String.fromCharCode(...bytes.slice(0, 4)) === "RIFF" &&
    String.fromCharCode(...bytes.slice(8, 12)) === "WEBP"
  ) {
    return "image/webp";
  }
  return null;
}
```

Paso 12.2 · Subir y servir la portada

Decisiones de diseño:

- **Solo el bibliotecario** puede subirla (`assertRole`) y se comprueba el contenido real antes de guardar.
- Se guarda como **base64 en Firestore** (colección `bookCovers`): para este proyecto evita configurar Firebase Storage. El límite de 700 000 caracteres respeta el máximo de 1 MiB por documento.
- La lectura (`GET`) es **pública** y con caché de un año: la URL lleva `?v=<coverVersion>`, así que cada cambio genera una URL nueva y la caché nunca sirve una imagen vieja.
- Subir una portada **actualiza** `updatedAt` **del libro**, de modo que la sincronización incremental la entrega a todos los dispositivos.
- Se escribe una línea de trazabilidad en el log del servidor (quién, qué libro y cuánto pesó), sin guardar la imagen en el log.

▸ **CREAR · library_api/app/api/books/[id]/cover/route.ts**

```
import { getCover, saveCover } from "@/lib/data/books";
import { assertRole, requireUser } from "@/lib/guard";
import { parseBody, route } from "@/lib/http";
import { ApiError } from "@/lib/errors";
import { sniffImageType } from "@/lib/image";
import { coverSchema } from "@/lib/schemas";

/** Las portadas son públicas (permiten usar <Image uri> sin cabeceras); la URL lleva ?v= para invalidar caché. */
export const GET = route(async (_request, ctx: RouteContext<"/api/books/[id]/cover">) => {
  const { id } = await ctx.params;
  const { contentType, bytes } = await getCover(id);
  return new Response(new Uint8Array(bytes), {
    headers: {
      "Content-Type": contentType,
      "Cache-Control": "public, max-age=31536000, immutable",
    },
  });
});

/**
 * PUT /api/books/:id/cover  { contentType, data(base64) }
 * Semana 12: la app envía la foto tomada con la cámara/galería. Solo bibliotecario.
 */
export const PUT = route(async (request, ctx: RouteContext<"/api/books/[id]/cover">) => {
  const user = await requireUser(request);
  assertRole(user, "librarian");
  const { id } = await ctx.params;
  const { contentType, data } = await parseBody(request, coverSchema);
  // No se confía en el Content-Type declarado: se verifica con los bytes reales.
  const real = sniffImageType(Buffer.from(data, "base64"));
  if (real !== contentType) {
    throw new ApiError(
      400,
      "VALIDATION_ERROR",
      "El archivo no es una imagen JPEG, PNG o WebP válida",
    );
  }
  console.info(`[cover] ${user.uid} actualizó portada de ${id} (${data.length} chars base64)`);
  return Response.json({ book: await saveCover(id, contentType, data) });
});
```

Paso 12.3 · Prueba de la detección de imágenes

▸ **CREAR · library_api/tests/image.test.ts**

```
import { describe, expect, it } from "vitest";
import { sniffImageType } from "@/lib/image";

const bytes = (...n: number[]) => Uint8Array.from(n);

describe("sniffImageType", () => {
  it("reconoce JPEG, PNG y WebP por sus magic bytes", () => {
    expect(sniffImageType(bytes(0xff, 0xd8, 0xff, 0xe0))).toBe("image/jpeg");
    expect(sniffImageType(bytes(0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a, 0x1a, 0x0a))).toBe("image/png");
    const webp = Uint8Array.from(
      [..."RIFF"]
        .map((c) => c.charCodeAt(0))
        .concat(
          [0, 0, 0, 0],
          [..."WEBP"].map((c) => c.charCodeAt(0)),
        ),
    );
    expect(sniffImageType(webp)).toBe("image/webp");
  });

  it("rechaza HTML u otros contenidos disfrazados de imagen", () => {
    expect(sniffImageType(new TextEncoder().encode("<html><script>alert(1)</script>"))).toBeNull();
    expect(sniffImageType(new Uint8Array())).toBeNull();
  });
});
```

> **Nota. Punto de control de la semana 12.** `pnpm typecheck` y `pnpm test` en verde (5 archivos de pruebas, 31 pruebas).

Idea. En un proyecto real con muchas imágenes lo correcto es **Firebase Storage** (o un CDN) y guardar solo la URL en Firestore. El base64 en la base de datos es una simplificación didáctica.

6.2 Cómo probar

```
# Con el token del bibliotecario (BIB): sube un PNG pequeño
curl -s -X PUT http://localhost:3000/api/books/clean-code/cover \
  -H "Authorization: Bearer $BIB" -H 'Content-Type: application/json' \
  -d "{\"contentType\":\"image/png\",\"data\":\"$(base64 -w0 portada.png)\"}"
# 403 con el token de un estudiante; 400 si el archivo no es una imagen real
curl -s http://localhost:3000/api/books/clean-code      # coverUrl cambia (?v=1)
```

6.3 Evidencia y criterios

Evidencia para el docente. Subida de portada desde la app (cámara) y desde la web; rechazo de archivos falsos y de estudiantes; la portada visible en ambos clientes.

- ☐ Un archivo HTML renombrado como `.png` es rechazado con `400`
- ☐ Un estudiante recibe `403`
- ☐ Tras subir, `coverUrl` cambia (`?v=` aumenta) y el libro aparece en `updatedSince`

7. El sitio web

El sitio ofrece las mismas funciones que la app usando **Server Components** (se renderizan en el servidor, sin exponer credenciales) y **Server Actions** (formularios que ejecutan código del servidor). Reutiliza exactamente las funciones de datos y reglas que construiste en las semanas 9 a 12.

Función	App móvil	Sitio web
Registro e inicio de sesión	Pantallas `sign-in` / `sign-up`	`/register` y `/login`
Sesión	`expo-secure-store`	Cookie `httpOnly`
Listar y buscar libros	SQLite local + sincronización	`/books?q=&category=` (consulta directa)
Detalle del libro	`book/[id]`	`/books/[id]`
Solicitar préstamo	Cola offline	Acción del servidor (necesita conexión)
Mis préstamos / gestión	Pestaña Mis préstamos o Gestión	`/loans`
Cambiar portada	Cámara o galería	Selector de archivo
Funciona sin conexión	**Sí**	No

7.1 Sesión del sitio web

La web no guarda tokens en el navegador: usa una **cookie de sesión** `httpOnly` (JavaScript no puede leerla), creada con `createSessionCookie` de Firebase. La capa de acceso a datos (DAL) la verifica en cada página y acción con `requireWebUser()`.

Paso 13.1 · Cookie y usuario de la sesión

▸ **CREAR · library_api/lib/web/session.ts**

```
import "server-only";
import { cache } from "react";
import { cookies } from "next/headers";
import { redirect } from "next/navigation";
import { verifySessionCookie, SESSION_MAX_AGE_MS } from "../services/auth";
import type { AuthUser, Role } from "../types";

export const SESSION_COOKIE = "library_session";

export async function setSessionCookie(value: string): Promise<void> {
  (await cookies()).set(SESSION_COOKIE, value, {
    httpOnly: true, // inaccesible para JavaScript del navegador
    secure: process.env.NODE_ENV === "production",
    sameSite: "lax",
    path: "/",
    maxAge: SESSION_MAX_AGE_MS / 1000,
  });
}

export async function clearSessionCookie(): Promise<void> {
  (await cookies()).delete(SESSION_COOKIE);
}

/** Usuario de la sesión web o null. `cache` evita verificar la cookie varias veces por petición. */
export const getWebUser = cache(async (): Promise<AuthUser | null> => {
  const value = (await cookies()).get(SESSION_COOKIE)?.value;
  if (!value) return null;
  try {
    return await verifySessionCookie(value);
  } catch {
    return null;
  }
});
```

```
/** Capa de acceso a datos (DAL): cada página/acción pide el usuario aquí, no confía solo en proxy.ts. */
export async function requireWebUser(role?: Role): Promise<AuthUser> {
  const user = await getWebUser();
  if (!user) redirect("/login");
  if (role && user.role !== role) redirect("/books");
  return user;
}
```

Paso 13.2 · Acciones del servidor

Cada acción hace lo mismo que la API: valida, comprueba el rol (`requireWebUser("student")`: el botón oculto en la interfaz **no** es una protección) y usa las mismas funciones de datos. `redirect()` lanza una excepción interna, por eso va **fuera** del `try/catch`.

▸ **CREAR · library_api/app/actions.ts**

```
"use server";

import { randomUUID } from "node:crypto";
import { redirect } from "next/navigation";
import { revalidatePath } from "next/cache";
import { saveCover } from "@/lib/data/books";
import { requestLoan, transitionLoan } from "@/lib/data/loans";
import { ApiError } from "@/lib/errors";
import { sniffImageType } from "@/lib/image";
import {
  loanActionSchema,
  loginSchema,
  registerSchema,
  MAX_COVER_BASE64_LENGTH,
} from "@/lib/schemas";
import { createSessionCookie, login, register } from "@/lib/services/auth";
import { clearSessionCookie, requireWebUser, setSessionCookie } from "@/lib/web/session";

export interface FormState {
  error?: string;
  ok?: string;
}

function messageOf(err: unknown): string {
  if (err instanceof ApiError) return err.message;
  console.error("[web] error no controlado:", err);
  return "Ocurrió un error inesperado. Inténtalo de nuevo.";
}

export async function loginAction(_prev: FormState, formData: FormData): Promise<FormState> {
  const parsed = loginSchema.safeParse(Object.fromEntries(formData));
  if (!parsed.success) return { error: parsed.error.issues[0]?.message ?? "Datos inválidos" };

  try {
    const { tokens } = await login(parsed.data.email, parsed.data.password);
    await setSessionCookie(await createSessionCookie(tokens.idToken));
  } catch (err) {
    return { error: messageOf(err) };
  }
  redirect("/books"); // redirect lanza una excepción: va fuera del try/catch
}

export async function registerAction(_prev: FormState, formData: FormData): Promise<FormState> {
  const parsed = registerSchema.safeParse(Object.fromEntries(formData));
  if (!parsed.success) return { error: parsed.error.issues[0]?.message ?? "Datos inválidos" };

  try {
    const { email, password, displayName } = parsed.data;
    const { tokens } = await register(email, password, displayName);
    await setSessionCookie(await createSessionCookie(tokens.idToken));
  } catch (err) {
    return { error: messageOf(err) };
  }
  redirect("/books");
}

export async function logoutAction(): Promise<void> {
  await clearSessionCookie();
  redirect("/login");
```

```
}

export async function requestLoanAction(bookId: string): Promise<FormState> {
  const user = await requireWebUser("student");
  try {
    const { created } = await requestLoan(user, bookId, randomUUID());
    revalidatePath("/loans");
    return {
      ok: created ? "Solicitud enviada. El bibliotecario la revisará." : "La solicitud ya existía.",
    };
  } catch (err) {
    return { error: messageOf(err) };
  }
}

export async function loanTransitionAction(loanId: string, action: string): Promise<FormState> {
  await requireWebUser("librarian");
  const parsed = loanActionSchema.safeParse({ action });
  if (!parsed.success) return { error: "Acción inválida" };
  try {
    await transitionLoan(loanId, parsed.data.action);
    revalidatePath("/loans");
    revalidatePath("/books");
    return { ok: "Actualizado" };
  } catch (err) {
    return { error: messageOf(err) };
  }
}

/** Límite crudo para que el base64 resultante (≈ 4/3) quepa en MAX_COVER_BASE64_LENGTH. */
const MAX_COVER_BYTES = Math.floor((MAX_COVER_BASE64_LENGTH * 3) / 4) - 4;

export async function uploadCoverAction(
  bookId: string,
  _prev: FormState,
  formData: FormData,
): Promise<FormState> {
  await requireWebUser("librarian");
  const file = formData.get("cover");
  if (!(file instanceof File) || file.size === 0) return { error: "Selecciona una imagen" };
  if (file.size > MAX_COVER_BYTES) {
    return { error: `La imagen supera ${Math.round(MAX_COVER_BYTES / 1024)} KB` };
  }

  const bytes = new Uint8Array(await file.arrayBuffer());
  const type = sniffImageType(bytes);
  if (!type) return { error: "El archivo debe ser una imagen JPEG, PNG o WebP" };

  try {
    await saveCover(bookId, type, Buffer.from(bytes).toString("base64"));
    revalidatePath(`/books/${bookId}`);
    revalidatePath("/books");
    return { ok: "Portada actualizada" };
  } catch (err) {
    return { error: messageOf(err) };
  }
}
```

7.2 Componentes de interfaz

Cinco archivos pequeños y reutilizables:

▸ **CREAR · library_api/components/ui.tsx**

```
import type { LoanStatus } from "@/lib/types";

export const inputClass =
  "w-full rounded-lg border border-zinc-300 bg-white px-3 py-2 text-sm text-zinc-900 " +
  "outline-none focus:border-indigo-500 focus:ring-2 focus:ring-indigo-200 " +
  "dark:border-zinc-700 dark:bg-zinc-900 dark:text-zinc-100";

export const buttonClass =
  "inline-flex items-center justify-center rounded-lg bg-indigo-600 px-4 py-2 text-sm " +
  "font-medium text-white transition hover:bg-indigo-700 " +
  "disabled:cursor-not-allowed disabled:opacity-50";
```

```
export const secondaryButtonClass =
  "inline-flex items-center justify-center rounded-lg border border-zinc-300 px-3 py-1.5 " +
  "text-sm font-medium text-zinc-700 transition hover:bg-zinc-100 disabled:opacity-50 " +
  "dark:border-zinc-700 dark:text-zinc-200 dark:hover:bg-zinc-800";

/** Recuadro de "no hay nada que mostrar" (listas vacías). */
export const emptyBoxClass =
  "rounded-xl border border-dashed border-zinc-300 p-8 text-center text-zinc-500 " +
  "dark:border-zinc-700";

const STATUS: Record<LoanStatus, { label: string; className: string }> = {
  requested: {
    label: "Solicitado",
    className: "bg-amber-100 text-amber-800 dark:bg-amber-900/40 dark:text-amber-200",
  },
  approved: {
    label: "Prestado",
    className: "bg-emerald-100 text-emerald-800 dark:bg-emerald-900/40 dark:text-emerald-200",
  },
  rejected: {
    label: "Rechazado",
    className: "bg-red-100 text-red-800 dark:bg-red-900/40 dark:text-red-200",
  },
  returned: {
    label: "Devuelto",
    className: "bg-zinc-200 text-zinc-700 dark:bg-zinc-800 dark:text-zinc-300",
  },
};

export function StatusBadge({ status }: { status: LoanStatus }) {
  const { label, className } = STATUS[status];
  return (
    <span className={`rounded-full px-2.5 py-0.5 text-xs font-medium ${className}`}>{label}</span>
  );
}

export function Alert({ kind, children }: { kind: "error" | "ok"; children: React.ReactNode }) {
  const styles =
    kind === "error"
      ? "bg-red-50 text-red-800 dark:bg-red-950 dark:text-red-200"
      : "bg-emerald-50 text-emerald-800 dark:bg-emerald-950 dark:text-emerald-200";
  return (
    <p
      role={kind === "error" ? "alert" : "status"}
      className={`rounded-lg px-3 py-2 text-sm ${styles}`}>
      {children}
    </p>
  );
}

export function formatDate(iso: string | null): string {
  if (!iso) return "—";
  return new Date(iso).toLocaleDateString("es-CO", {
    day: "2-digit",
    month: "short",
    year: "numeric",
  });
}
```

▸ **CREAR · library_api/components/header.tsx**

```
import Link from "next/link";
import { logoutAction } from "@/app/actions";
import { getWebUser } from "@/lib/web/session";
import { secondaryButtonClass } from "./ui";

export async function Header() {
  const user = await getWebUser();

  return (
    <header className="border-b border-zinc-200 dark:border-zinc-800">
      <div className="mx-auto flex h-14 max-w-5xl items-center justify-between px-4">
        <Link href={user ? "/books" : "/login"} className="font-semibold tracking-tight">
          Biblioteca
        </Link>
        {user && (
          <nav className="flex items-center gap-4 text-sm">
            <Link href="/books" className="hover:underline">
```

```
              Libros
            </Link>
            <Link href="/loans" className="hover:underline">
              {user.role === "librarian" ? "Gestión de préstamos" : "Mis préstamos"}
            </Link>
            <span className="hidden text-zinc-500 sm:inline">
              {user.name} · {user.role === "librarian" ? "Bibliotecario" : "Estudiante"}
            </span>
            <form action={logoutAction}>
              <button className={secondaryButtonClass}>Salir</button>
            </form>
          </nav>
        )}
      </div>
    </header>
  );
}
```

▸ **CREAR · library_api/components/auth-form.tsx**

```
"use client";

import Link from "next/link";
import { useActionState } from "react";
import { loginAction, registerAction, type FormState } from "@/app/actions";
import { Alert, buttonClass, inputClass } from "./ui";

const initial: FormState = {};

export function AuthForm({ mode }: { mode: "login" | "register" }) {
  const [state, action, pending] = useActionState(
    mode === "login" ? loginAction : registerAction,
    initial,
  );
  const isLogin = mode === "login";

  return (
    <form action={action} className="flex flex-col gap-4">
      {!isLogin && (
        <label className="flex flex-col gap-1 text-sm font-medium">
          Nombre
          <input
            name="displayName"
            required
            minLength={2}
            autoComplete="name"
            className={inputClass}
          />
        </label>
      )}
      <label className="flex flex-col gap-1 text-sm font-medium">
        Correo
        <input name="email" type="email" required autoComplete="email" className={inputClass} />
      </label>
      <label className="flex flex-col gap-1 text-sm font-medium">
        Contraseña
        <input
          name="password"
          type="password"
          required
          minLength={isLogin ? 1 : 6}
          autoComplete={isLogin ? "current-password" : "new-password"}
          className={inputClass}
        />
      </label>
      {state.error && <Alert kind="error">{state.error}</Alert>}
      <button type="submit" disabled={pending} className={buttonClass}>
        {pending ? "Procesando…" : isLogin ? "Iniciar sesión" : "Crear cuenta de estudiante"}
      </button>
      <p className="text-center text-sm text-zinc-600 dark:text-zinc-400">
        {isLogin ? (
          <>
            ¿No tienes cuenta?{" "}
            <Link href="/register" className="text-indigo-600 underline">
              Regístrate
            </Link>
          </>
        ) : (
```

```
          <>
            ¿Ya tienes cuenta?{" "}
            <Link href="/login" className="text-indigo-600 underline">
              Inicia sesión
            </Link>
          </>
        )}
      </p>
    </form>
  );
}
```

▸ **CREAR · library_api/components/loan-controls.tsx**

```
"use client";

import { useActionState } from "react";
import {
  loanTransitionAction,
  requestLoanAction,
  uploadCoverAction,
  type FormState,
} from "@/app/actions";
import { Alert, buttonClass, inputClass, secondaryButtonClass } from "./ui";

const initial: FormState = {};

/** Estudiante: botón "Solicitar préstamo". */
export function RequestLoanButton({ bookId, disabled }: { bookId: string; disabled: boolean }) {
  const [state, action, pending] = useActionState(() => requestLoanAction(bookId), initial);
  return (
    <form action={action} className="flex flex-col gap-3">
      <button type="submit" disabled={disabled || pending} className={buttonClass}>
        {pending ? "Enviando…" : disabled ? "Sin ejemplares disponibles" : "Solicitar préstamo"}
      </button>
      {state.error && <Alert kind="error">{state.error}</Alert>}
      {state.ok && <Alert kind="ok">{state.ok}</Alert>}
    </form>
  );
}

const LABELS = { approve: "Prestar", reject: "Rechazar", return: "Registrar devolución" } as const;

/** Bibliotecario: botones según el estado del préstamo. */
export function LoanActions({
  loanId,
  actions,
}: {
  loanId: string;
  actions: (keyof typeof LABELS)[];
}) {
  const [state, action, pending] = useActionState(
    (_prev: FormState, formData: FormData) =>
      loanTransitionAction(loanId, String(formData.get("action"))),
    initial,
  );
  return (
    <form action={action} className="flex flex-col items-end gap-2">
      <div className="flex gap-2">
        {actions.map((a) => (
          <button
            key={a}
            name="action"
            value={a}
            disabled={pending}
            className={a === "approve" ? buttonClass : secondaryButtonClass}>
            {LABELS[a]}
          </button>
        ))}
      </div>
      {state.error && <Alert kind="error">{state.error}</Alert>}
    </form>
  );
}

/** Bibliotecario: subir/cambiar portada (equivalente web de la cámara en la app). */
export function CoverUploadForm({ bookId }: { bookId: string }) {
  const [state, action, pending] = useActionState(uploadCoverAction.bind(null, bookId), initial);
```

```
  return (
    <form action={action} className="flex flex-col gap-3">
      <label className="flex flex-col gap-1 text-sm font-medium">
        Cambiar portada (JPEG, PNG o WebP, máx. ~500 KB)
        <input
          name="cover"
          type="file"
          accept="image/jpeg,image/png,image/webp"
          required
          className={inputClass}
        />
      </label>
      <button type="submit" disabled={pending} className={secondaryButtonClass}>
        {pending ? "Subiendo…" : "Subir portada"}
      </button>
      {state.error && <Alert kind="error">{state.error}</Alert>}
      {state.ok && <Alert kind="ok">{state.ok}</Alert>}
    </form>
  );
}
```

▸ **CREAR · library_api/components/book-card.tsx**

```
import Link from "next/link";
import type { Book } from "@/lib/types";

const placeholderClass =
  "flex items-center justify-center bg-indigo-100 text-4xl font-bold text-indigo-600 " +
  "dark:bg-indigo-950 dark:text-indigo-300";

const cardClass =
  "flex gap-4 rounded-xl border border-zinc-200 p-4 transition " +
  "hover:border-indigo-400 hover:shadow-sm dark:border-zinc-800";

export function BookCover({ book, className = "" }: { book: Book; className?: string }) {
  if (book.coverUrl) {
    // Imagen dinámica servida por la propia API; next/image no aporta aquí.
    return (
      // eslint-disable-next-line @next/next/no-img-element
      <img
        src={book.coverUrl}
        alt={`Portada de ${book.title}`}
        className={`object-cover ${className}`}
      />
    );
  }
  return (
    <div aria-hidden className={`${placeholderClass} ${className}`}>
      {book.title.charAt(0)}
    </div>
  );
}

export function BookCard({ book }: { book: Book }) {
  const available = book.availableCopies > 0;
  return (
    <Link href={`/books/${book.id}`} className={cardClass}>
      <BookCover book={book} className="h-28 w-20 shrink-0 rounded-md" />
      <div className="flex min-w-0 flex-col gap-1">
        <h2 className="font-semibold leading-snug">{book.title}</h2>
        <p className="text-sm text-zinc-600 dark:text-zinc-400">{book.author}</p>
        <p className="text-xs text-zinc-500">
          {book.category} · {book.year}
        </p>
        <p
          className={`mt-auto text-xs font-medium ${available ? "text-emerald-600" : "text-red-600"}`}>
          {available
            ? `${book.availableCopies} de ${book.totalCopies} disponibles`
            : "Sin ejemplares"}
        </p>
      </div>
    </Link>
  );
}
```

7.3 Estilos, layout y redirección

Reemplaza los estilos globales (usa la fuente del sistema y define el tema claro/oscuro):

▸ **CREAR · library_api/app/globals.css**

```
@import "tailwindcss";

:root {
  --background: #ffffff;
  --foreground: #171717;
}

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
}

@media (prefers-color-scheme: dark) {
  :root {
    --background: #0a0a0a;
    --foreground: #ededed;
  }
}

body {
  background: var(--background);
  color: var(--foreground);
  font-family: var(--font-geist-sans), system-ui, sans-serif;
}
```

El layout raíz lee la cookie para mostrar el nombre del usuario, así que envuelve el encabezado en `<Suspense>` (regla de `cacheComponents`: lo que depende de la petición no puede bloquear el resto de la página):

▸ **CREAR · library_api/app/layout.tsx**

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import { Suspense } from "react";
import { Header } from "@/components/header";
import "./globals.css";

const geistSans = Geist({
  variable: "--font-geist-sans",
  subsets: ["latin"],
});

const geistMono = Geist_Mono({
  variable: "--font-geist-mono",
  subsets: ["latin"],
});

export const metadata: Metadata = {
  title: { default: "Biblioteca", template: "%s · Biblioteca" },
  description: "Catálogo y préstamo de libros. API y sitio web del proyecto de Desarrollo Móvil.",
};

export default function RootLayout({ children }: LayoutProps<"/">) {
  return (
    <html lang="es" className={`${geistSans.variable} ${geistMono.variable} h-full antialiased`}>
      <body className="min-h-full flex flex-col">
        {/* La cookie de sesión es dato de petición: se lee dentro de Suspense para no bloquear el resto. */}
        <Suspense fallback={<div className="h-14 border-b border-zinc-200 dark:border-zinc-800" />}>
          <Header />
        </Suspense>
        <main className="mx-auto w-full max-w-5xl flex-1 px-4 py-8">{children}</main>
      </body>
    </html>
  );
}
```

La página raíz redirige al catálogo:

▸ **CREAR · library_api/app/page.tsx**

```
import { redirect } from "next/navigation";

export default function Home() {
  redirect("/books");
}
```

`proxy.ts` hace una comprobación **optimista**: solo mira si existe la cookie para mandar rápido a `/login`. La verificación real ocurre en el DAL.

▸ **CREAR · library_api/proxy.ts**

```
import { NextResponse, type NextRequest } from "next/server";

const SESSION_COOKIE = "library_session";
const PUBLIC_PATHS = ["/login", "/register"];

/**
 * Comprobación OPTIMISTA: solo mira si existe la cookie para redirigir rápido.
 * La verificación real de la sesión ocurre en lib/web/session.ts (requireWebUser).
 * /api/* queda fuera (ver matcher): ahí la autenticación es por Bearer token.
 */
export function proxy(request: NextRequest) {
  const { pathname } = request.nextUrl;
  const hasSession = request.cookies.has(SESSION_COOKIE);
  const isPublic = PUBLIC_PATHS.includes(pathname);

  if (!hasSession && !isPublic) {
    return NextResponse.redirect(new URL("/login", request.url));
  }
  // Ojo: no se redirige de /login a /books aquí. Una cookie vencida provocaría un bucle
  // (/books → /login → /books). La página de login valida la sesión y redirige si es válida.
  return NextResponse.next();
}

export const config = {
  matcher: ["/((?!api|_next/static|_next/image|favicon.ico|.*\\.(?:svg|png|jpg|ico)$).*)"],
};
```

Ojo. En el proxy **no** se redirige de `/login` a `/books` cuando hay cookie. Una cookie vencida provocaría un bucle infinito: `/books` → `/login` → `/books`. De eso se encarga la propia página de login (comprueba la sesión y redirige si es válida).

7.4 Las páginas

Los archivos entre paréntesis (`(auth)`, `(app)`) son **grupos de rutas**: organizan carpetas sin cambiar la URL. Todas leen datos de la petición (cookies, parámetros), así que el contenido va dentro de `<Suspense>` con un skeleton mientras llega.

▸ **CREAR · library_api/app/(auth)/login/page.tsx**

```
import type { Metadata } from "next";
import { redirect } from "next/navigation";
import { Suspense } from "react";
import { AuthForm } from "@/components/auth-form";
import { getWebUser } from "@/lib/web/session";

export const metadata: Metadata = { title: "Iniciar sesión" };

async function RedirectIfSignedIn() {
  if (await getWebUser()) redirect("/books");
  return null;
}

export default function LoginPage() {
  return (
```

```
    <div className="mx-auto max-w-sm">
      <Suspense>
        <RedirectIfSignedIn />
      </Suspense>
      <h1 className="mb-6 text-2xl font-semibold">Iniciar sesión</h1>
      <AuthForm mode="login" />
    </div>
  );
}
```

▸ **CREAR · library_api/app/(auth)/register/page.tsx**

```
import type { Metadata } from "next";
import { redirect } from "next/navigation";
import { Suspense } from "react";
import { AuthForm } from "@/components/auth-form";
import { getWebUser } from "@/lib/web/session";

export const metadata: Metadata = { title: "Crear cuenta" };

async function RedirectIfSignedIn() {
  if (await getWebUser()) redirect("/books");
  return null;
}

export default function RegisterPage() {
  return (
    <div className="mx-auto max-w-sm">
      <Suspense>
        <RedirectIfSignedIn />
      </Suspense>
      <h1 className="mb-6 text-2xl font-semibold">Crear cuenta</h1>
      <AuthForm mode="register" />
    </div>
  );
}
```

▸ **CREAR · library_api/app/(app)/books/page.tsx**

```
import type { Metadata } from "next";
import { Suspense } from "react";
import { BookCard } from "@/components/book-card";
import { buttonClass, emptyBoxClass, inputClass } from "@/components/ui";
import { listBooks, listCategories } from "@/lib/data/books";
import { requireWebUser } from "@/lib/web/session";

export const metadata: Metadata = { title: "Libros" };

const first = (v: string | string[] | undefined) => (Array.isArray(v) ? v[0] : v);

async function BookList({ searchParams }: { searchParams: PageProps<"/books">["searchParams"] }) {
  await requireWebUser();
  const params = await searchParams;
  const q = first(params.q)?.trim().slice(0, 100) || undefined;
  const category = first(params.category)?.trim().slice(0, 60) || undefined;

  const [books, categories] = await Promise.all([listBooks({ q, category }), listCategories()]);

  return (
    <>
      <form className="mb-6 flex flex-col gap-3 sm:flex-row" role="search">
        <input
          name="q"
          defaultValue={q}
          placeholder="Buscar por título o autor"
          className={inputClass}
          aria-label="Buscar"
        />
        <select
          name="category"
          defaultValue={category ?? ""}
          className={`${inputClass} sm:w-48`}
          aria-label="Categoría">
          <option value="">Todas las categorías</option>
          {categories.map((c) => (
            <option key={c} value={c}>
              {c}
```

```
            </option>
          ))}
        </select>
        <button className={buttonClass}>Buscar</button>
      </form>

      {books.length === 0 ? (
        <p className={emptyBoxClass}>No encontramos libros con esos criterios.</p>
      ) : (
        <ul className="grid gap-4 sm:grid-cols-2">
          {books.map((b) => (
            <li key={b.id}>
              <BookCard book={b} />
            </li>
          ))}
        </ul>
      )}
    </>
  );
}

function Skeleton() {
  return (
    <div className="grid gap-4 sm:grid-cols-2" aria-busy>
      {Array.from({ length: 4 }, (_, i) => (
        <div key={i} className="h-36 animate-pulse rounded-xl bg-zinc-100 dark:bg-zinc-900" />
      ))}
    </div>
  );
}

export default function BooksPage(props: PageProps<"/books">) {
  return (
    <>
      <h1 className="mb-6 text-2xl font-semibold">Catálogo de libros</h1>
      <Suspense fallback={<Skeleton />}>
        <BookList searchParams={props.searchParams} />
      </Suspense>
    </>
  );
}
```

▸ **CREAR · library_api/app/(app)/books/[id]/page.tsx**

```
import type { Metadata } from "next";
import Link from "next/link";
import { notFound } from "next/navigation";
import { Suspense } from "react";
import { BookCover } from "@/components/book-card";
import { CoverUploadForm, RequestLoanButton } from "@/components/loan-controls";
import { getBook } from "@/lib/data/books";
import { ApiError } from "@/lib/errors";
import { requireWebUser } from "@/lib/web/session";

export const metadata: Metadata = { title: "Detalle del libro" };

async function BookDetail({ params }: { params: PageProps<"/books/[id]">["params"] }) {
  const user = await requireWebUser();
  const { id } = await params;

  const book = await getBook(id).catch((err) => {
    if (err instanceof ApiError && err.status === 404) notFound();
    throw err;
  });

  return (
    <article className="grid gap-8 md:grid-cols-[14rem_1fr]">
      <BookCover book={book} className="aspect-[3/4] w-56 rounded-xl" />
      <div className="flex flex-col gap-4">
        <div>
          <h1 className="text-3xl font-semibold tracking-tight">{book.title}</h1>
          <p className="mt-1 text-zinc-600 dark:text-zinc-400">{book.author}</p>
        </div>
        <dl className="grid grid-cols-3 gap-4 text-sm">
          <div>
            <dt className="text-zinc-500">Categoría</dt>
            <dd className="font-medium">{book.category}</dd>
          </div>
```

```
          <div>
            <dt className="text-zinc-500">Año</dt>
            <dd className="font-medium">{book.year}</dd>
          </div>
          <div>
            <dt className="text-zinc-500">Disponibles</dt>
            <dd className="font-medium">
              {book.availableCopies} de {book.totalCopies}
            </dd>
          </div>
        </dl>
        <p className="leading-relaxed">{book.description}</p>

        {user.role === "student" ? (
          <RequestLoanButton bookId={book.id} disabled={book.availableCopies <= 0} />
        ) : (
          <section className="rounded-xl border border-zinc-200 p-4 dark:border-zinc-800">
            <h2 className="mb-3 font-medium">Administración (bibliotecario)</h2>
            <CoverUploadForm bookId={book.id} />
          </section>
        )}
        <Link href="/books" className="text-sm text-indigo-600 underline">
          ← Volver al catálogo
        </Link>
      </div>
    </article>
  );
}

export default function BookPage(props: PageProps<"/books/[id]">) {
  return (
    <Suspense
      fallback={
        <div className="h-64 animate-pulse rounded-xl bg-zinc-100 dark:bg-zinc-900" aria-busy />
      }>
      <BookDetail params={props.params} />
    </Suspense>
  );
}
```

▸ **CREAR · library_api/app/(app)/loans/page.tsx**

```
import type { Metadata } from "next";
import Link from "next/link";
import { Suspense } from "react";
import { LoanActions } from "@/components/loan-controls";
import { StatusBadge, emptyBoxClass, formatDate } from "@/components/ui";
import { listLoans } from "@/lib/data/loans";
import type { Loan } from "@/lib/types";
import { requireWebUser } from "@/lib/web/session";

export const metadata: Metadata = { title: "Préstamos" };

const cardClass =
  "flex flex-col gap-3 rounded-xl border border-zinc-200 p-4 " +
  "sm:flex-row sm:items-center sm:justify-between dark:border-zinc-800";

function actionsFor(loan: Loan): ("approve" | "reject" | "return")[] {
  if (loan.status === "requested") return ["approve", "reject"];
  if (loan.status === "approved") return ["return"];
  return [];
}

async function LoanList() {
  const user = await requireWebUser();
  const loans = await listLoans(user);
  const librarian = user.role === "librarian";

  if (loans.length === 0) {
    return (
      <p className={emptyBoxClass}>
        {librarian ? "Aún no hay solicitudes." : "Todavía no has solicitado libros."}{" "}
        <Link href="/books" className="text-indigo-600 underline">
          Ver catálogo
        </Link>
      </p>
    );
  }
```

```
  return (
    <ul className="flex flex-col gap-3">
      {loans.map((loan) => (
        <li key={loan.id} className={cardClass}>
          <div className="flex flex-col gap-1">
            <Link href={`/books/${loan.bookId}`} className="font-semibold hover:underline">
              {loan.bookTitle}
            </Link>
            {librarian && (
              <p className="text-sm text-zinc-600 dark:text-zinc-400">
                {loan.userName} · {loan.userEmail}
              </p>
            )}
            <p className="text-xs text-zinc-500">
              Solicitado {formatDate(loan.requestedAt)}
              {loan.status === "approved" && ` · Entrega ${formatDate(loan.dueDate)}`}
              {loan.status === "returned" && ` · Devuelto ${formatDate(loan.returnedAt)}`}
            </p>
          </div>
          <div className="flex flex-col items-start gap-2 sm:items-end">
            <StatusBadge status={loan.status} />
            {librarian && actionsFor(loan).length > 0 && (
              <LoanActions loanId={loan.id} actions={actionsFor(loan)} />
            )}
          </div>
        </li>
      ))}
    </ul>
  );
}

export default function LoansPage() {
  return (
    <>
      <h1 className="mb-6 text-2xl font-semibold">Préstamos</h1>
      <Suspense
        fallback={
          <div className="h-40 animate-pulse rounded-xl bg-zinc-100 dark:bg-zinc-900" aria-busy />
        }>
        <LoanList />
      </Suspense>
    </>
  );
}
```

> **Nota. Punto de control del sitio web.** Además de `pnpm typecheck` y `pnpm test`, ejecuta `pnpm lint` y `pnpm build`: el build de Next es el que verifica las reglas de `cacheComponents` (por ejemplo, que no leas `cookies()` fuera de `Suspense`).

Como el contenido se transmite por partes (streaming), una URL inexistente como `/books/no-existe` responde `200` con la pantalla de "no encontrado": el código de estado ya se envió cuando se descubre que el libro no existe. Es el comportamiento esperado de este modelo de renderizado.

7.5 Cómo probar el sitio

```
pnpm dev
```

1. Abre `http://localhost:3000`: te lleva a `/login`.
2. Entra con tu **estudiante** (el que creaste en 4.4): ves el catálogo, buscas `colera`, abres un libro y pulsas **Solicitar préstamo**.
3. Cierra sesión y entra con tu **bibliotecario**: en Gestión de préstamos aprueba la solicitud; en el detalle de un libro sube una portada (JPEG, PNG o WebP de hasta ~500 KB).
4. Con una cookie falsa (`library_session=basura` en las herramientas del navegador) la página te devuelve a `/login` sin mostrar datos.

8. Pruebas finales y restablecer datos

8.1 Restablecer los préstamos y las portadas

Borra todos los préstamos, elimina las portadas y restaura los ejemplares (no toca usuarios). Úsalo entre pruebas:

▸ **CREAR · library_api/scripts/reset-demo.mts**

```
/**
 * Deja los datos de demostración como al inicio: borra TODOS los préstamos, restaura los ejemplares
 * disponibles y elimina las portadas. No toca usuarios.   pnpm reset-demo
 */
import { cert, initializeApp } from "firebase-admin/app";
import { getFirestore } from "firebase-admin/firestore";

initializeApp({
  credential: cert({
    projectId: process.env.FIREBASE_PROJECT_ID!,
    clientEmail: process.env.FIREBASE_CLIENT_EMAIL!,
    privateKey: (process.env.FIREBASE_PRIVATE_KEY ?? "").replace(/\\n/g, "\n"),
  }),
});
const db = getFirestore();

const loans = await db.collection("loans").get();
await Promise.all(loans.docs.map((d) => d.ref.delete()));

const covers = await db.collection("bookCovers").get();
await Promise.all(covers.docs.map((d) => d.ref.delete()));

const now = new Date().toISOString();
const books = await db.collection("books").get();
await Promise.all(
  books.docs.map((d) =>
    d.ref.update({ availableCopies: d.data().totalCopies, coverVersion: 0, updatedAt: now }),
  ),
);
console.log(
  `Restablecido: ${loans.size} préstamos y ${covers.size} portadas eliminados, ${books.size} libros restaurados`,
);
```

8.2 Prueba de humo contra Firebase real

Este script ejecuta **41 comprobaciones de extremo a extremo** contra la API en marcha y **limpia todo lo que crea**. No necesita cuentas fijas: crea un estudiante, otro estudiante y un bibliotecario **temporales** con contraseñas aleatorias, y los elimina al terminar junto con los préstamos y la portada de prueba:

Bloque	Qué comprueba
Autenticación (8)	Salud, `401` sin token, login correcto e incorrecto, roles, refresh y `me`
Catálogo (7)	10 libros, categorías, búsqueda sin tildes, filtro, detalle, `404`, sincronización incremental
Préstamos del estudiante (8)	Solicitar, **idempotencia**, `ALREADY_ACTIVE`, `404`, `400`, `403` por rol
Gestión del bibliotecario (7)	Aprobar, ejemplares que bajan y vuelven, devolución, sincronización incremental
Conflicto de ejemplares (7)	Registro público, correo repetido, **dos estudiantes y un solo ejemplar** (`NO_COPIES`), aislamiento entre usuarios
Portada (4)	`403` a estudiantes, archivo falso rechazado, subida y descarga idénticas

▸ **CREAR · library_api/scripts/smoke.mts**

```
/**
 * Prueba de humo de extremo a extremo contra la API en ejecución y Firebase REAL.
 *   1) pnpm seed            (una vez: carga los libros)
 *   2) pnpm dev  (o build + start)
 *   3) pnpm smoke           (BASE_URL=http://localhost:3000 por defecto)
 * No necesita cuentas fijas: crea un estudiante, otro estudiante y un bibliotecario TEMPORALES con
 * contraseñas aleatorias, y los elimina al terminar junto con los préstamos de prueba.
 */
import { randomUUID } from "node:crypto";
import { deflateSync, crc32 } from "node:zlib";
import { cert, initializeApp } from "firebase-admin/app";
import { getAuth } from "firebase-admin/auth";
import { getFirestore } from "firebase-admin/firestore";

const BASE = process.env.BASE_URL ?? "http://localhost:3000";
const RUN = Date.now();
/** Contraseña aleatoria distinta en cada ejecución: nunca queda escrita en ningún archivo. */
const PASSWORD = `Tmp-${randomUUID()}`;
const STUDENT_EMAIL = `smoke-student-${RUN}@example.test`;
const LIBRARIAN_EMAIL = `smoke-librarian-${RUN}@example.test`;

initializeApp({
  credential: cert({
    projectId: process.env.FIREBASE_PROJECT_ID!,
    clientEmail: process.env.FIREBASE_CLIENT_EMAIL!,
    privateKey: process.env.FIREBASE_PRIVATE_KEY!.replace(/\\n/g, "\n"),
  }),
});

let failures = 0;
function check(name: string, ok: boolean, extra?: unknown) {
  console.log(
    `${ok ? "  ✓" : "  ✗"} ${name}${ok || extra === undefined ? "" : `  → ${JSON.stringify(extra)}`}`,
  );
  if (!ok) failures++;
}

/** Forma mínima de las respuestas que se inspeccionan (las comprobaciones verifican los valores reales). */
interface Dto {
  error?: { code?: string; message?: string };
  user: { email: string; role: string };
  tokens: { idToken: string; refreshToken: string };
  books: { id: string; title: string; availableCopies: number; coverUrl: string | null }[];
  categories: string[];
  book: { id: string; availableCopies: number; coverUrl: string | null };
  loan: { id: string; status: string; dueDate: string | null };
  loans: { id: string; userEmail: string }[];
}
interface Res {
  status: number;
  body: Dto;
  headers: Headers;
}
async function call(
  method: string,
  path: string,
  opts: { token?: string; body?: unknown } = {},
): Promise<Res> {
  const res = await fetch(`${BASE}${path}`, {
    method,
    headers: {
      ...(opts.body !== undefined ? { "Content-Type": "application/json" } : {}),
      ...(opts.token ? { Authorization: `Bearer ${opts.token}` } : {}),
    },
    body: opts.body !== undefined ? JSON.stringify(opts.body) : undefined,
  });
  const text = await res.text();
  let body = {} as Dto;
  try {
    body = JSON.parse(text) as Dto;
  } catch {
    /* respuesta binaria */
  }
  return { status: res.status, body, headers: res.headers };
}
```

```
/** PNG sólido de 64x64 generado en memoria (sin dependencias). */
function makePng(): Buffer {
  const size = 64;
  const raw = Buffer.alloc((size * 3 + 1) * size);
  for (let y = 0; y < size; y++) {
    const row = y * (size * 3 + 1);
    for (let x = 0; x < size; x++) {
      raw[row + 1 + x * 3] = (x * 4) % 256;
      raw[row + 2 + x * 3] = (y * 4) % 256;
      raw[row + 3 + x * 3] = 120;
    }
  }
  const chunk = (type: string, data: Buffer) => {
    const len = Buffer.alloc(4);
    len.writeUInt32BE(data.length);
    const body = Buffer.concat([Buffer.from(type), data]);
    const crc = Buffer.alloc(4);
    crc.writeUInt32BE(crc32(body));
    return Buffer.concat([len, body, crc]);
  };
  const ihdr = Buffer.alloc(13);
  ihdr.writeUInt32BE(size, 0);
  ihdr.writeUInt32BE(size, 4);
  ihdr[8] = 8;
  ihdr[9] = 2;
  return Buffer.concat([
    Buffer.from([0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a, 0x1a, 0x0a]),
    chunk("IHDR", ihdr),
    chunk("IDAT", deflateSync(raw)),
    chunk("IEND", Buffer.alloc(0)),
  ]);
}

const createdLoanIds: string[] = [];
const touchedBooks = new Set<string>();
const tempEmail = `smoke-register-${RUN}@example.test`;

/** Crea un usuario temporal con un rol (Admin SDK, igual que `pnpm create-user`). */
async function createTempUser(email: string, displayName: string, role: "student" | "librarian") {
  const created = await getAuth().createUser({ email, password: PASSWORD, displayName });
  await getAuth().setCustomUserClaims(created.uid, { role });
  await getFirestore()
    .collection("users")
    .doc(created.uid)
    .set({ email, displayName, role, createdAt: new Date().toISOString() });
}

async function main() {
  console.log(`Probando ${BASE}\n`);
  await createTempUser(STUDENT_EMAIL, "Smoke Estudiante", "student");
  await createTempUser(LIBRARIAN_EMAIL, "Smoke Bibliotecario", "librarian");

  console.log("Salud y autenticación");
  check("GET /api/health → 200", (await call("GET", "/api/health")).status === 200);
  check("GET /api/books sin token → 401", (await call("GET", "/api/books")).status === 401);
  const bad = await call("POST", "/api/auth/login", {
    body: { email: STUDENT_EMAIL, password: "incorrecta" },
  });
  check(
    "login con contraseña errónea → 401 INVALID_CREDENTIALS",
    bad.status === 401 && bad.body.error?.code === "INVALID_CREDENTIALS",
    bad.body,
  );

  const stu = await call("POST", "/api/auth/login", {
    body: { email: ` ${STUDENT_EMAIL.toUpperCase()} `, password: PASSWORD },
  });
  check("login estudiante (correo con espacios/mayúsculas) → 200", stu.status === 200, stu.body);
  const student = stu.body.tokens.idToken as string;
  check("rol del estudiante = student", stu.body.user.role === "student", stu.body.user);

  const lib = await call("POST", "/api/auth/login", {
    body: { email: LIBRARIAN_EMAIL, password: PASSWORD },
  });
  check(
    "login bibliotecario → 200 con rol librarian",
    lib.status === 200 && lib.body.user.role === "librarian",
    lib.body.user,
```

```
);
const librarian = lib.body.tokens.idToken as string;

const ref = await call("POST", "/api/auth/refresh", {
  body: { refreshToken: stu.body.tokens.refreshToken },
});
check(
  "refresh token → nuevo idToken",
  ref.status === 200 && !!ref.body.tokens?.idToken,
  ref.body,
);
check(
  "GET /api/auth/me",
  (await call("GET", "/api/auth/me", { token: student })).body.user?.email === STUDENT_EMAIL,
);

console.log("\nCatálogo");
const all = await call("GET", "/api/books", { token: student });
check("hay 10 libros", all.body.books?.length === 10, all.body.books?.length);
check("devuelve categorías", all.body.categories?.length >= 3, all.body.categories);
const colera = await call("GET", "/api/books?q=colera", { token: student });
check(
  "búsqueda sin tildes 'colera' encuentra 'El amor en los tiempos del cólera'",
  colera.body.books?.length === 1,
  colera.body.books?.map((b) => b.title),
);
const tech = await call("GET", "/api/books?category=Tecnolog%C3%ADa", { token: student });
check(
  "filtro por categoría Tecnología → 3",
  tech.body.books?.length === 3,
  tech.body.books?.length,
);
const detail = await call("GET", "/api/books/rayuela", { token: student });
check(
  "detalle de 'rayuela' con 1 ejemplar",
  detail.status === 200 && detail.body.book.availableCopies === 1,
  detail.body,
);
check(
  "libro inexistente → 404",
  (await call("GET", "/api/books/no-existe", { token: student })).status === 404,
);
const future = await call(
  "GET",
  `/api/books?updatedSince=${encodeURIComponent(new Date(Date.now() + 60_000).toISOString())}`,
  { token: student },
);
check("sync incremental: updatedSince futuro → 0 libros", future.body.books?.length === 0);

console.log("\nPréstamos (estudiante)");
const rid = `smoke-${Date.now()}-a`;
const first = await call("POST", "/api/loans", {
  token: student,
  body: { bookId: "clean-code", clientRequestId: rid },
});
check(
  "solicitar 'clean-code' → 201 requested",
  first.status === 201 && first.body.loan.status === "requested",
  first.body,
);
createdLoanIds.push(first.body.loan?.id);
touchedBooks.add("clean-code");
const again = await call("POST", "/api/loans", {
  token: student,
  body: { bookId: "clean-code", clientRequestId: rid },
});
check(
  "reintento con el mismo clientRequestId es idempotente → 200 y mismo id",
  again.status === 200 && again.body.loan.id === first.body.loan.id,
  again.body,
);
const dup = await call("POST", "/api/loans", {
  token: student,
  body: { bookId: "clean-code", clientRequestId: `${rid}-otro` },
});
check(
  "segunda solicitud activa del mismo libro → 409 ALREADY_ACTIVE",
  dup.status === 409 && dup.body.error?.code === "ALREADY_ACTIVE",
```

```
  dup.body,
);
const missing = await call("POST", "/api/loans", {
  token: student,
  body: { bookId: "no-existe", clientRequestId: `${rid}-x1` },
});
check("libro inexistente → 404", missing.status === 404);
check(
  "clientRequestId inválido → 400",
  (
    await call("POST", "/api/loans", {
      token: student,
      body: { bookId: "clean-code", clientRequestId: "../x" },
    })
  ).status === 400,
);
check(
  "estudiante no puede aprobar → 403",
  (
    await call("PATCH", `/api/loans/${first.body.loan.id}`, {
      token: student,
      body: { action: "approve" },
    })
  ).status === 403,
);
check(
  "bibliotecario no puede solicitar → 403",
  (
    await call("POST", "/api/loans", {
      token: librarian,
      body: { bookId: "clean-code", clientRequestId: `${rid}-lib` },
    })
  ).status === 403,
);
const mine = await call("GET", "/api/loans", { token: student });
check(
  "GET /api/loans (estudiante) solo trae los suyos",
  mine.body.loans.every((l) => l.userEmail === STUDENT_EMAIL),
);

console.log("\nGestión (bibliotecario)");
const all2 = await call("GET", "/api/loans?status=requested", { token: librarian });
check(
  "bibliotecario ve la solicitud pendiente",
  all2.body.loans.some((l) => l.id === first.body.loan.id),
  all2.body.loans?.length,
);
const ok = await call("PATCH", `/api/loans/${first.body.loan.id}`, {
  token: librarian,
  body: { action: "approve" },
});
check(
  "aprobar → approved con fecha de entrega",
  ok.status === 200 && ok.body.loan.status === "approved" && !!ok.body.loan.dueDate,
  ok.body,
);
const afterApprove = await call("GET", "/api/books/clean-code", { token: student });
check(
  "ejemplares disponibles bajaron 3 → 2",
  afterApprove.body.book.availableCopies === 2,
  afterApprove.body.book.availableCopies,
);
const delta = await call(
  "GET",
  `/api/books?updatedSince=${encodeURIComponent(new Date(Date.now() - 30_000).toISOString())}`,
  { token: student },
);
check(
  "sync incremental trae solo el libro modificado",
  delta.body.books?.length === 1 && delta.body.books[0].id === "clean-code",
  delta.body.books?.map((b) => b.id),
);
check(
  "aprobar dos veces → 409 INVALID_STATE",
  (
    await call("PATCH", `/api/loans/${first.body.loan.id}`, {
      token: librarian,
      body: { action: "approve" },
```

```
    })
  ).body.error?.code === "INVALID_STATE",
);
const ret = await call("PATCH", `/api/loans/${first.body.loan.id}`, {
  token: librarian,
  body: { action: "return" },
});
check("devolver → returned", ret.body.loan?.status === "returned", ret.body);
check(
  "ejemplares vuelven a 3",
  (await call("GET", "/api/books/clean-code", { token: student })).body.book.availableCopies ===
    3,
);

console.log("\nConflicto 'sin ejemplares' (el caso de la cola offline)");
const reg = await call("POST", "/api/auth/register", {
  body: { email: tempEmail, password: PASSWORD, displayName: "Estudiante Temporal" },
});
check(
  "registro público crea estudiante → 201",
  reg.status === 201 && reg.body.user.role === "student",
  reg.body,
);
const student2 = reg.body.tokens?.idToken as string;
check(
  "registro con correo repetido → 409 EMAIL_IN_USE",
  (
    await call("POST", "/api/auth/register", {
      body: { email: tempEmail, password: PASSWORD, displayName: "Otra" },
    })
  ).body.error?.code === "EMAIL_IN_USE",
);
const l1 = await call("POST", "/api/loans", {
  token: student,
  body: { bookId: "rayuela", clientRequestId: `${rid}-r1` },
});
createdLoanIds.push(l1.body.loan?.id);
touchedBooks.add("rayuela");
const l2 = await call("POST", "/api/loans", {
  token: student2,
  body: { bookId: "rayuela", clientRequestId: `${rid}-r2` },
});
createdLoanIds.push(l2.body.loan?.id);
check(
  "dos estudiantes piden el último ejemplar (ambas se aceptan como solicitud)",
  l1.status === 201 && l2.status === 201,
  [l1.status, l2.status],
);
check(
  "el bibliotecario aprueba la primera",
  (
    await call("PATCH", `/api/loans/${l1.body.loan.id}`, {
      token: librarian,
      body: { action: "approve" },
    })
  ).status === 200,
);
const fail = await call("PATCH", `/api/loans/${l2.body.loan.id}`, {
  token: librarian,
  body: { action: "approve" },
});
check(
  "aprobar la segunda → 409 NO_COPIES (consistencia garantizada por transacción)",
  fail.status === 409 && fail.body.error?.code === "NO_COPIES",
  fail.body,
);
const l3 = await call("POST", "/api/loans", {
  token: student2,
  body: { bookId: "rayuela", clientRequestId: `${rid}-r3` },
});
check(
  "solicitar un libro sin ejemplares → 409 (ya hay solicitud activa o NO_COPIES)",
  l3.status === 409,
  l3.body,
);
check(
  "estudiante 2 no ve préstamos ajenos por id → 404",
  (await call("GET", `/api/loans/${l1.body.loan.id}`, { token: student2 })).status === 404,
```

```
  );

  console.log("\nPortada (semana 12)");
  const png = makePng().toString("base64");
  check(
    "estudiante no puede subir portada → 403",
    (
      await call("PUT", "/api/books/clean-code/cover", {
        token: student,
        body: { contentType: "image/png", data: png },
      })
    ).status === 403,
  );
  const fake = Buffer.from("<html><script>alert(1)</script>".padEnd(200, " ")).toString("base64");
  check(
    "archivo que no es imagen (aunque diga image/png) → 400",
    (
      await call("PUT", "/api/books/clean-code/cover", {
        token: librarian,
        body: { contentType: "image/png", data: fake },
      })
    ).status === 400,
  );
  const up = await call("PUT", "/api/books/clean-code/cover", {
    token: librarian,
    body: { contentType: "image/png", data: png },
  });
  check(
    "bibliotecario sube portada PNG → 200 con coverUrl",
    up.status === 200 && !!up.body.book.coverUrl,
    up.body,
  );
  const img = await fetch(`${BASE}${up.body.book.coverUrl}`);
  check(
    "GET de la portada → image/png con los mismos bytes",
    img.headers.get("content-type") === "image/png" &&
      Buffer.from(await img.arrayBuffer()).toString("base64") === png,
  );
}

async function cleanup() {
  console.log("\nLimpieza");
  const db = getFirestore();
  for (const id of createdLoanIds.filter(Boolean)) await db.collection("loans").doc(id).delete();
  for (const id of touchedBooks) {
    const snap = await db.collection("books").doc(id).get();
    if (!snap.exists) continue;
    await db.collection("bookCovers").doc(id).delete();
    await snap.ref.update({
      availableCopies: snap.data()!.totalCopies,
      coverVersion: 0,
      updatedAt: new Date().toISOString(),
    });
  }
  for (const email of [STUDENT_EMAIL, LIBRARIAN_EMAIL, tempEmail]) {
    try {
      const u = await getAuth().getUserByEmail(email);
      await getAuth().deleteUser(u.uid);
      await db.collection("users").doc(u.uid).delete();
    } catch {
      /* no se llegó a crear */
    }
  }
  const loans = createdLoanIds.filter(Boolean).length;
  console.log(
    `  ${loans} préstamos, ${touchedBooks.size} libros restaurados, usuarios temporales eliminados`,
  );
}

try {
  await main();
} catch (err) {
  failures++;
  console.error("\nError inesperado:", err);
} finally {
  await cleanup();
}
console.log(failures === 0 ? "\n Todo OK" : `\n ${failures} comprobación(es) fallaron`);
```

```
process.exit(failures === 0 ? 0 : 1);

pnpm dev                      # en una terminal
pnpm smoke                    # en otra (BASE_URL=http://localhost:3000 por defecto)
pnpm reset-demo               # deja los datos como al inicio
```

Debe terminar con `Todo OK`.

8.3 Verificación completa

▸ **TERMINAL · library_api**

```
pnpm typecheck
pnpm test
pnpm lint
pnpm build
```

> **Nota. Punto de control final.** Los cuatro comandos sin errores. La API y el sitio web están completos: ahora sigue con la **Guía 2** para construir la app móvil.

9. Seguridad y despliegue

9.1 Lista de verificación antes de publicar

- ☐ `.env` y el `.json` de la cuenta de servicio **no** están en git (`git status`, `git log -p`)
- ☐ Reglas de Firestore en `allow … if false`
- ☐ En el Firebase de producción no hay cuentas de prueba con contraseñas simples o conocidas (revisa Authentication → Usuarios)
- ☐ La API se sirve por **HTTPS** y la app usa esa URL
- ☐ Las variables de entorno están configuradas en el servicio de despliegue, no en el código
- ☐ `pnpm build`, `pnpm test`, `pnpm lint` y `pnpm typecheck` sin errores

9.2 Desplegar en Vercel (resumen)

1. Sube el repositorio a GitHub **sin** `.env` ni el `.json`.
2. En Vercel: Add New → Project, elige el repositorio (detecta Next.js solo).
3. En Settings → Environment Variables crea las cuatro variables. Para `FIREBASE_PRIVATE_KEY` pega el valor con los saltos de línea escritos como `\n`.
4. Despliega. La URL `https://…vercel.app` es la que irá en la app (`EXPO_PUBLIC_API_URL`).

10. Problemas frecuentes

Síntoma	Causa probable	Solución
`CONFIGURATION_NOT_FOUND` al iniciar sesión	Authentication no está inicializado	Consola → Authentication → Comenzar y habilitar correo/contraseña
`SERVICE_DISABLED` o `PERMISSION_DENIED` en Firestore	No se creó la base de datos o su API está desactivada	Consola → Firestore → Crear base de datos (puede tardar unos minutos en propagarse)
`Faltan variables de entorno: …`	`.env` incompleto o clave privada sin comillas	Revisa `.env.example`; la clave va entre comillas con `\n`
`Top-level await is currently not supported` al ejecutar scripts	Un script `.ts` se interpreta como CommonJS	Los scripts son `.mts` (ESM) y se ejecutan con `tsx`
`ERR_PNPM_IGNORED_BUILDS`	pnpm 11 pide decidir sobre scripts de instalación	Crea `pnpm-workspace.yaml` **antes** de instalar, como en el paso 2.3
El teléfono no llega a la API	La API escucha solo en `localhost` o hay firewall	`pnpm dev:lan`, misma red Wi-Fi, abrir el puerto 3000 (en WSL2 ver la Guía 2)
Un bibliotecario nuevo aparece como estudiante	Su token tiene el rol anterior	Cerrar sesión y volver a entrar
`409 ALREADY_ACTIVE`	Ya hay una solicitud o préstamo activo del libro	Es una regla de negocio, no un error
El build de Next falla por `cookies()` o `headers()`	Se leyó un dato de la petición fuera de `Suspense`	Mueve la lectura a un componente dentro de `<Suspense>`
`pnpm typecheck` dice que no existe `RouteContext` o `PageProps`	Aún no se generaron los tipos de rutas	Ejecuta `pnpm typecheck` (corre `next typegen` primero)

11. Bitácora de uso de IA generativa

Registra cada uso: propósito, qué obtuviste, cómo lo **validaste** (pruebas, documentación, ejecución) y qué decidiste. Nunca pegues credenciales ni el contenido del `.env` en un asistente.

Fecha	Herramienta y prompt (resumido)	Qué respondió	Cómo lo validé	Decisión
(ej.) Sem. 10	"¿Cómo verifico un idToken de Firebase en Next.js?"	Sugirió decodificar el JWT a mano	Lo contrasté con la documentación del Admin SDK: decodificar no verifica la firma	Rechazada; usé `verifyIdToken`
(ej.) Sem. 11	"Escribe pruebas para la regla de ejemplares"	6 casos	Los ejecuté con `pnpm test`; agregué el caso del límite de ejemplares que faltaba	Aceptada con ajustes

12. Referencias

- Next.js. Route Handlers, Proxy y Authentication. Documentación incluida en `node_modules/next/dist/docs/`.
- Firebase. Admin SDK: verify ID tokens, custom claims, session cookies. https://firebase.google.com/docs/auth/admin
- Firebase. Auth REST API. https://firebase.google.com/docs/reference/rest/auth
- Firebase. Transactions (Firestore). https://firebase.google.com/docs/firestore/manage-data/transactions
- Zod. https://zod.dev
- OWASP. Application Security Verification Standard (ASVS) y MASVS. https://mas.owasp.org
- UNESCO (2023). Guidance for generative AI in education and research. https://unesdoc.unesco.org/ark:/48223/pf0000386693